In-Class Solutions

Instructor key | Algorithms & Data Structures

Every solution in this key was run in Python 3.13 against the LeetCode examples and extra edge cases, and compared with a slower reference solution on random inputs where one exists. Every memory figure was formed from a formula and then checked against tracemalloc at the stated input size.

How to use this key

- **Credit rule.** An answer earns extra credit only if the student can explain and defend it. Ask why it works, why the complexity is what they wrote, and what changes for a different input.
- **Complexity convention.** Auxiliary space is the extra memory beyond the input, including anything that is built and returned. The key lists the output explicitly when it matters.
- **Memory estimates are estimates.** Accept any answer within about 30% of the key when the student states sensible assumptions (a different n, a different input shape, a different Python version). Grade the method: list pointers plus object sizes, correct n, input counted separately from auxiliary.
- **Alternatives.** Each problem lists other approaches with their complexity. Accept them if they are analyzed correctly.
- **Effort rule.** A student asking for help must show 2 failed attempts in the box on the answer sheet.

Memory reference

Object	Approximate size (CPython 3.13, 64-bit)
int	28 bytes (-5 to 256 shared)
float	24 bytes
str	41 bytes + 1 per ASCII character
list	56 bytes + 8 per slot (about 12% spare room if built with append)
tuple	40 bytes + 8 per item
dict	about 50 bytes per entry at 100,000 entries (37 at 1,000; 184 for a tiny dict)
set	about 42 bytes per element at 100,000 (33 at 1,000)
deque	about 8.3 bytes per element + 600 overhead
node object	about 100 bytes
call frame	about 150 bytes per active call

A set or dict grows by resizing, so its peak can be about 1.5 times its final size. Recursion frames are estimated, not measured.

Class 1: Big-O and Python memory
Problem A (easier)

#1480 Running Sum of 1d Array

Easy. https://leetcode.com/problems/running-sum-of-1d-array/

Given a list of integers nums, return a list where entry i is the sum of nums[0] through nums[i] (the running sum).

```
Example: nums = [1, 2, 3, 4]  ->  [1, 3, 6, 10]
```

Solution (what goes on the board)

```
def runningSum(nums):
    for i in range(1, len(nums)):
        nums[i] = nums[i] + nums[i - 1]
    return nums
```

Reasoning, step by step

1. Entry 0 is already correct: its running sum is itself.
2. For each later position i, the running sum is the previous running sum plus nums[i]. The previous entry already holds that total, so add it in.
3. Work left to right so nums[i - 1] is finished before you use it.
4. Return the same list. It was changed in place, so no second list is built.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: one loop of n - 1 steps, each a constant amount of work. Space: only the index i and a few temporary ints; the result is the input list itself.

Memory estimate

Assumptions. n = 100,000 distinct ints above 256 (so none are cached small ints). The sums grow, so each update creates one new 28-byte int and frees the old one.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	index, range iterator and one temporary int, about 150 bytes	150 bytes
Total	input + auxiliary	3.4 MB

Each update replaces an old int with a new one of the same size, so memory does not grow. (A tool such as tracemalloc shows a few hundred KB of allocator churn; that is not growth.)

Other approaches

Approach	Time	Space
Build a new list with a loop and append	O(n)	O(n)
itertools.accumulate (not for the board)	O(n)	O(n)

Class 1: Big-O and Python memory
Problem B (harder)

#217 Contains Duplicate

Easy. https://leetcode.com/problems/contains-duplicate/

Given a list of integers nums, return True if any value appears at least twice, and False if every value is distinct.

```
Example: nums = [1, 2, 3, 1]  ->  True        nums = [1, 2, 3, 4]  ->  False
```

Solution (what goes on the board)

```
    def containsDuplicate(nums):
        seen = set()
        for x in nums:
            if x in seen:
                return True
            seen.add(x)
        return False
```

Reasoning, step by step

1. Brute force: compare every pair. That is about n(n-1)/2 comparisons, so O(n^2) time and O(1) space.
2. Better idea: remember what we have seen. A set answers "have I seen x?" in O(1) on average.
3. Walk the list once. If x is already in the set we found a duplicate; otherwise add x.
4. If the loop ends, every value was new, so return False.
5. We traded O(n) memory for going from O(n^2) to O(n) time.

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: n steps, each an O(1) average set lookup and insert. Space: in the worst case (all distinct) the set holds n values.

Memory estimate

Assumptions. n = 100,000 distinct ints above 256. The set stores references to the ints that already exist in the input, so it creates no new int objects. A set that large uses a table of 2^17 slots of 32 bytes.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	set table of about 42 bytes per element at this size	4.0 MB
Total	input + auxiliary	7.4 MB

A set grows by resizing into a bigger table, so the peak during the last resize is about 1.5 times the final table (measured: about 6 MB).

Other approaches

Approach	Time	Space
Brute force: two nested loops	O(n^2)	O(1)
Sort, then compare neighbors	O(n log n)	O(1) to O(n)
Set (the solution above)	O(n)	O(n)

Class 2: Arrays and in-place updates
Problem A (easier)

#283 Move Zeroes

Easy. https://leetcode.com/problems/move-zeroes/

Given a list nums, move all 0s to the end while keeping the order of the non-zero values. You must change the list in place, without making a copy.

```
Example: nums = [0, 1, 0, 3, 12]  ->  [1, 3, 12, 0, 0]
```

Solution (what goes on the board)

```
    def moveZeroes(nums):
        write = 0
        for read in range(len(nums)):
            if nums[read] != 0:
                nums[write], nums[read] = nums[read], nums[write]
                write += 1
```

Reasoning, step by step

1. Use two indexes. read scans every position. write marks where the next non-zero value belongs.
2. Everything before write is already finished: non-zero values in their original order.
3. When nums[read] is non-zero, swap it into position write and move write forward by one.
4. Zeros are never moved forward on purpose; they get pushed back by the swaps.
5. The function returns nothing. The list itself is changed.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: read visits each position once, with at most one swap per step. Space: two index variables.

Memory estimate

Assumptions. n = 100,000 ints, half of them 0. The swap only moves references around.

Part	How it is counted	Estimate
Input	56 + 8n + 28n/2 (zeros are the shared small int 0)	2.1 MB
Auxiliary	two indexes and a range iterator, about 150 bytes	150 bytes
Total	input + auxiliary	2.1 MB

Class 2: Arrays and in-place updates
Problem B (harder)

#88 Merge Sorted Array

Easy. https://leetcode.com/problems/merge-sorted-array/

nums1 has m real values in sorted order followed by n zeros of spare room (length m + n). nums2 has n values in sorted order. Merge nums2 into nums1 so that nums1 is sorted. Change nums1 in place.

```
Example: nums1 = [1, 2, 3, 0, 0, 0], m = 3, nums2 = [2, 5, 6], n = 3  ->
nums1 = [1, 2, 2, 3, 5, 6]
```

Solution (what goes on the board)

```
    def merge(nums1, m, nums2, n):
        i = m - 1
        j = n - 1
        k = m + n - 1
        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[k] = nums1[i]
                i -= 1
            else:
                nums1[k] = nums2[j]
                j -= 1
            k -= 1
```

Reasoning, step by step

1. Merging from the front would overwrite values of nums1 that we still need.
2. The empty room is at the back, so fill from the back: write the largest remaining value into position k.
3. i points at the last unused value of nums1, j at the last unused value of nums2.
4. Compare nums1[i] and nums2[j], write the larger at k, and move that pointer and k left.
5. Stop when nums2 is used up. Whatever is left of nums1 is already in place.

Complexity

Time	O(m + n)
Auxiliary space	O(1)

Time: each step writes one position, and there are m + n positions at most (fewer, since we stop when nums2 is used). Space: three index variables.

Memory estimate

Assumptions. m = n = 50,000, so nums1 has 100,000 slots and nums2 has 50,000. Values are distinct ints above 256.

Part	How it is counted	Estimate
Input	(56 + 8n + 28n) for nums1 + (56 + 8n/2 + 28n/2) for nums2	5.1 MB
Auxiliary	three indexes, about 150 bytes	150 bytes
Total	input + auxiliary	5.2 MB

Class 3: Strings and prefix sums
Problem A (easier)

#14 Longest Common Prefix

Easy. https://leetcode.com/problems/longest-common-prefix/

Given a list of strings strs, return the longest string that is a prefix of every string. If there is none, return the empty string "".

```
Example: strs = ["flower", "flow", "flight"]  ->  "fl"        strs = ["dog", "racecar", "car"]  ->  ""
```

Solution (what goes on the board)

```
    def longestCommonPrefix(strs):
        first = strs[0]
        for i in range(len(first)):
            for s in strs:
                if i == len(s) or s[i] != first[i]:
                    return first[:i]
        return first
```

Reasoning, step by step

1. The answer can be no longer than the first string, so use the first string as the template.
2. Scan column by column: for position i, check that every string has the same character as first at i.
3. The first time any string is too short or has a different character at i, the prefix is first[:i].
4. If we get through all of first without a mismatch, the whole first string is the prefix.
5. Slicing happens once, at the end, so the copying cost is paid once.

Complexity

Time	O(S) where S = total characters in all strings (at most m x L)
Auxiliary space	O(L) for the returned prefix, L <= shortest string

Time: in the worst case every character of every string is looked at once, so it is proportional to the total number of characters. Space: only the returned slice, which is at most one string long.

Memory estimate

Assumptions. m = 200 strings of 200 characters each, all identical in the worst case (constraints allow up to 200 strings of up to 200 characters). A str costs about 41 bytes + 1 byte per ASCII character.

Part	How it is counted	Estimate
Input	200 strings x (41 + 200) + list of 200 pointers (56 + 8 x 200)	49 KB
Auxiliary	the returned slice: 41 + 200 bytes, plus a few indexes	341 bytes
Total	input + auxiliary	49 KB

Other approaches

Approach	Time	Space
Compare all strings to a running prefix and shorten it	O(S)	O(L)
Sort the strings; compare only first and last	O(S + m log m)	O(m)

Class 3: Strings and prefix sums
Problem B (harder)

#303 Range Sum Query - Immutable

Easy. https://leetcode.com/problems/range-sum-query-immutable/

Design a class NumArray. It is built once from a list nums. Then sumRange(left, right) is called many times and must return the sum of nums[left] through nums[right], inclusive.

```
Example: NumArray([-2, 0, 3, -5, 2, -1]);  sumRange(0, 2) -> 1;  sumRange(2, 5) -> -1;  sumRange(0, 5) -> -3
```

Solution (what goes on the board)

```
class NumArray:
    def __init__(self, nums):
        self.prefix = [0]
        for x in nums:
            self.prefix.append(self.prefix[-1] + x)

    def sumRange(self, left, right):
        return self.prefix[right + 1] - self.prefix[left]
```

Reasoning, step by step

1. Summing the range on every call costs O(n) per query, which is slow if there are many queries.
2. Do the work once: prefix[i] holds the sum of the first i numbers, with prefix[0] = 0.
3. Build it in one pass: each entry is the previous entry plus the next number.
4. The sum of nums[left..right] is everything up to right, minus everything before left: prefix[right + 1] - prefix[left].
5. The extra 0 at the front means left = 0 needs no special case.

Complexity

Time	Build O(n); each sumRange O(1)
Auxiliary space	O(n)

Time: one pass of n steps to build; each query is two list lookups and one subtraction. Space: the prefix list holds n + 1 numbers.

Memory estimate

Assumptions. n = 10,000 values (the LeetCode limit), large enough that every prefix sum is a new 28-byte int above 256.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	prefix list: 56 + 8(n + 1) + 28(n + 1)	352 KB
Total	input + auxiliary	703 KB

Class 4: Hash sets and hash maps
Problem A (easier)

#242 Valid Anagram

Easy. https://leetcode.com/problems/valid-anagram/

Given two strings s and t of lowercase letters, return True if t is an anagram of s (it uses exactly the same letters with the same counts), and False otherwise.

```
Example: s = "anagram", t = "nagaram"  ->  True        s = "rat", t = "car"
->  False
```

Solution (what goes on the board)

```
    def isAnagram(s, t):
        if len(s) != len(t):
            return False
        counts = {}
        for ch in s:
            counts[ch] = counts.get(ch, 0) + 1
        for ch in t:
            if counts.get(ch, 0) == 0:
                return False
            counts[ch] -= 1
        return True
```

Reasoning, step by step

1. If the lengths differ, the answer is False at once.
2. Count each letter of s in a dict: letter -> how many times it appears.
3. Walk through t and use up one count per letter. If a letter has no count left, t has too many of it, so return False.
4. Because the lengths are equal and no count went below zero, every count ends at zero: True.
5. The dict has at most 26 keys, so its size does not grow with n.

Complexity

Time	O(n)
Auxiliary space	O(1) (at most 26 keys)

Time: two passes over strings of length n, each step an O(1) dict operation. Space: the dict holds at most 26 letters, however long the strings are.

Memory estimate

Assumptions. n = 50,000 characters in each string (the LeetCode limit). The dict has 26 keys; the 1-character strings are shared singletons, and the counts above 256 are new 28-byte ints.

Part	How it is counted	Estimate
Input	2 strings x (41 + n)	98 KB
Auxiliary	dict with 26 keys (about 1.2 KB) + 26 counts x 28 bytes	2.0 KB
Total	input + auxiliary	100 KB

Other approaches

Approach	Time	Space
Sort both strings and compare	O(n log n)	O(n)
Fixed list of 26 counters	O(n)	O(1)

Class 4: Hash sets and hash maps
Problem B (harder)

#1 Two Sum

Easy. https://leetcode.com/problems/two-sum/

Given a list of integers nums and an integer target, return the indexes of the two numbers that add up to target. Exactly one such pair exists, and you may not use the same position twice.

```
Example: nums = [2, 7, 11, 15], target = 9  ->  [0, 1]
```

Solution (what goes on the board)

```
    def twoSum(nums, target):
        seen = {}
        for i, x in enumerate(nums):
            need = target - x
            if need in seen:
                return [seen[need], i]
            seen[x] = i
```

Reasoning, step by step

1. Brute force: try every pair, O(n^2) time and O(1) space.
2. For each x the partner we need is target - x. Instead of searching for it, remember every earlier value in a dict: value -> index.
3. At position i compute need = target - x. If need is already in the dict, return its index and i.
4. Otherwise store x with its index and keep going. Store after checking so we never pair x with itself.
5. One pass, one dict: O(n) time and O(n) space.

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: one pass with O(1) average dict operations. Space: in the worst case (the pair is at the very end) the dict holds n - 1 entries.

Memory estimate

Assumptions. n = 10,000 distinct ints above 256 (the LeetCode limit), with the answer at the very end so the dict grows to n entries. Keys point to existing ints; each stored index i is a new 28-byte int.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	dict at about 52 bytes per entry + 28 bytes per stored index	781 KB
Total	input + auxiliary	1.1 MB

Other approaches

Approach	Time	Space
Brute force: all pairs	O(n^2)	O(1)

Approach	Time	Space
Sort indexes by value, then two pointers	O(n log n)	O(n)

Class 5: Two pointers
Problem A (easier)

#125 Valid Palindrome

Easy. https://leetcode.com/problems/valid-palindrome/

A phrase is a palindrome if, after making every letter lowercase and removing every character that is not a letter or digit, it reads the same forward and backward. Given a string s, return True if it is a palindrome.

```
Example: s = "A man, a plan, a canal: Panama"  ->  True        s = "race a car"  ->  False
```

Solution (what goes on the board)

```
    def isPalindrome(s):
        left = 0
        right = len(s) - 1
        while left < right:
            while left < right and not s[left].isalnum():
                left += 1
            while left < right and not s[right].isalnum():
                right -= 1
            if s[left].lower() != s[right].lower():
                return False
            left += 1
            right -= 1
        return True
```

Reasoning, step by step

1. Put one pointer at each end and move them toward each other.
2. Skip any character that is not a letter or digit: advance left or retreat right until both point at real characters.
3. Compare the two characters ignoring case. A mismatch means False at once.
4. On a match move both pointers inward. When they meet or cross, every pair matched: True.
5. Cleaning the string first and comparing with its reverse also works but copies the string, costing O(n) extra space.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: each pointer only moves inward, so together they make at most n moves. Space: two indexes (single-character strings from s[i] and lower() are cached, so they cost nothing).

Memory estimate

Assumptions. n = 200,000 characters (the LeetCode limit), all letters, in a palindrome (the worst case, checked all the way to the middle). A str costs about 41 bytes + 1 byte per ASCII character.

Part	How it is counted	Estimate
Input	41 + n	195 KB

Part	How it is counted	Estimate
Auxiliary	two indexes and a few temporaries, about 150 bytes	150 bytes
Total	input + auxiliary	195 KB

Other approaches

Approach	Time	Space
Clean the string, compare with its reverse	O(n)	O(n)

Class 5: Two pointers
Problem B (harder)

#977 Squares of a Sorted Array

Easy. https://leetcode.com/problems/squares-of-a-sorted-array/

Given a list nums sorted in non-decreasing order (it may contain negative numbers), return a new list of the squares of every number, also sorted in non-decreasing order.

`Example: nums = [-4, -1, 0, 3, 10]  ->  [0, 1, 9, 16, 100]`

Solution (what goes on the board)

```
    def sortedSquares(nums):
        n = len(nums)
        result = [0] * n
        left = 0
        right = n - 1
        for k in range(n - 1, -1, -1):
            if abs(nums[left]) > abs(nums[right]):
                result[k] = nums[left] * nums[left]
                left += 1
            else:
                result[k] = nums[right] * nums[right]
                right -= 1
        return result
```

Reasoning, step by step

1. Squaring and then sorting costs O(n log n). We can do better because the input is already sorted.
2. The largest square is at one of the two ends: a big negative or a big positive number.
3. Put pointers at both ends. Compare |nums[left]| and |nums[right]|; the bigger one is the largest remaining square.
4. Write that square at the back of the result (position k, from n - 1 down to 0) and move that pointer inward.
5. Every position of result is written once, from largest to smallest.

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: n steps, each moving one pointer. Space: the result list of n squares (the squares are new ints; the pointers cost almost nothing).

Memory estimate

Assumptions. n = 10,000 values (the LeetCode limit), values in the thousands so every square is a new 28-byte int above 256. The result is counted as auxiliary space.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	result list: 56 + 8n + 28n	352 KB
Total	input + auxiliary	703 KB

Other approaches

Approach	Time	Space
Square every value, then sorted()	O(n log n)	O(n)

Class 6: Sliding window
Problem A (easier)

#643 Maximum Average Subarray I

Easy. https://leetcode.com/problems/maximum-average-subarray-i/

Given a list nums of integers and an integer k, find the contiguous subarray of length k with the largest average, and return that average.

```
Example: nums = [1, 12, -5, -6, 50, 3], k = 4  ->  12.75   (the window [12, -5, -6, 50])
```

Solution (what goes on the board)

```
    def findMaxAverage(nums, k):
        window = 0
        for i in range(k):
            window += nums[i]
        best = window
        for i in range(k, len(nums)):
            window += nums[i] - nums[i - k]
            if window > best:
                best = window
        return best / k
```

Reasoning, step by step

1. The window with the largest average has the largest sum, since k is fixed. So find the largest sum of k in a row, then divide by k at the end.
2. Brute force sums every window from scratch: about n x k steps.
3. Sum the first k numbers once.
4. To slide the window one step right, add the new number nums[i] and subtract the number leaving, nums[i - k].
5. Keep the best sum seen. Return best / k.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: k steps for the first window and n - k slides of constant work each. Space: window, best and i.

Memory estimate

Assumptions. n = 100,000 ints above 256 (the LeetCode limit is 100,000 values); the running sum is a new int each step but only one is alive at a time.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	a few ints and a float, about 150 bytes	150 bytes
Total	input + auxiliary	3.4 MB

Other approaches

Approach	Time	Space
Brute force: re-sum each window	O(nk)	O(1)
Prefix sums: window = prefix[i+k] - prefix[i]	O(n)	O(n)

Class 6: Sliding window
Problem B (harder)

#121 Best Time to Buy and Sell Stock

Easy. https://leetcode.com/problems/best-time-to-buy-and-sell-stock/

prices[i] is the price of a stock on day i. Choose one day to buy and a later day to sell. Return the largest possible profit, or 0 if no profit is possible.

```
Example: prices = [7, 1, 5, 3, 6, 4]  ->  5   (buy at 1, sell at 6)
prices = [7, 6, 4, 3, 1]  ->  0
```

Solution (what goes on the board)

```
    def maxProfit(prices):
        left = 0
        best = 0
        for right in range(1, len(prices)):
            if prices[right] < prices[left]:
                left = right
            else:
                best = max(best, prices[right] - prices[left])
        return best
```

Reasoning, step by step

1. Think of the window [left, right]: left is the day we buy, right is the day we sell.
2. Brute force tries every buy and sell pair: O(n^2).
3. Move right across the days. If today is cheaper than the buy day, a better buy day has been found: set left = right.
4. Otherwise today is a possible sell day: profit = prices[right] - prices[left]; keep the largest.
5. The window never moves left, so one pass is enough.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: right visits each day once. Space: left, right and best.

Memory estimate

Assumptions. n = 100,000 prices (the LeetCode limit) above 256. Only a handful of extra ints are alive at once.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	left, right, best and one temporary, about 150 bytes	150 bytes
Total	input + auxiliary	3.4 MB

Other approaches

Approach	Time	Space
Brute force: all buy/sell pairs	O(n^2)	O(1)

Class 7: Stacks
Problem A (easier)

#20 Valid Parentheses

Easy. https://leetcode.com/problems/valid-parentheses/

Given a string s containing only the characters ( ) [ ] { }, return True if it is valid: every opening bracket is closed by the same type of bracket, and brackets close in the correct order.

```
Example: s = "()[]{}"  ->  True        s = "(]"  ->  False        s = "([)]"
->  False
```

Solution (what goes on the board)

```
    def isValid(s):
        stack = []
        pairs = {')': '(', ']': '[', '}': '{'}
        for ch in s:
            if ch in pairs:
                if not stack or stack.pop() != pairs[ch]:
                    return False
            else:
                stack.append(ch)
        return not stack
```

Reasoning, step by step

1. The most recent opening bracket must be closed first. That is last-in, first-out: a stack.
2. Push every opening bracket.
3. On a closing bracket, pop the stack. If the stack is empty, or the popped bracket is the wrong type, return False.
4. The dict pairs maps each closer to the opener it needs.
5. At the end the stack must be empty, otherwise some opener was never closed.

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: each character is pushed or popped once. Space: in the worst case (all openers) the stack holds n characters.

Memory estimate

Assumptions. n = 10,000 characters (the LeetCode limit), all "(" (the worst case for the stack). Single characters are shared strings, so the stack holds only pointers. The 3-entry dict costs about 180 bytes.

Part	How it is counted	Estimate
Input	41 + n	9.8 KB
Auxiliary	stack list 8 bytes per pointer (about 12% extra growth room) + dict 180 bytes	88 KB
Total	input + auxiliary	98 KB

Other approaches

Approach	Time	Space
Repeatedly delete "()", "[]", "{}" from the string	O(n^2)	O(n)

Class 7: Stacks
Problem B (harder)

#844 Backspace String Compare

Easy. https://leetcode.com/problems/backspace-string-compare/

Two strings s and t contain lowercase letters and the character #, which means backspace (it deletes the previous character, and does nothing if there is none). Return True if the two strings are equal after all backspaces are applied.

```
Example: s = "ab#c", t = "ad#c"  ->  True   (both become "ac")        s =
"a#c", t = "b"  ->  False
```

Solution (what goes on the board)

```
    def backspaceCompare(s, t):
        def build(text):
            stack = []
            for ch in text:
                if ch == '#':
                    if stack:
                        stack.pop()
                else:
                    stack.append(ch)
            return stack
        return build(s) == build(t)
```

Reasoning, step by step

1. Typing characters and then backspacing is exactly a stack: a letter is pushed, # pops the most recent letter.
2. Write a helper build(text) that returns the final stack of letters.
3. A # on an empty stack does nothing.
4. Build both stacks and compare them with ==.
5. Follow-up: with two pointers moving from the end of each string you can do it in O(1) extra space.

Complexity

Time	O(n + m)
Auxiliary space	O(n + m)

Time: each character is processed once, and the final comparison is at most min(n, m) steps. Space: each stack can hold up to the full length of its string.

Memory estimate

Assumptions. n = m = 200 characters (the LeetCode limit), no # characters (the worst case for stack size). Single characters are shared, so the stacks hold pointers only.

Part	How it is counted	Estimate
Input	2 strings x (41 + n)	482 bytes
Auxiliary	two lists of n pointers, 8 bytes each, plus 56 byte headers and growth room	3.6 KB
Total	input + auxiliary	4.1 KB

Other approaches

Approach	Time	Space
Two pointers from the end, skipping deleted letters	O(n + m)	O(1)

Class 8: Queues and deques
Problem A (easier)

#933 Number of Recent Calls

Easy. https://leetcode.com/problems/number-of-recent-calls/

Design a class RecentCounter. ping(t) records a request at time t (in milliseconds; each t is larger than the one before) and returns how many requests were made in the last 3000 ms, that is, in the range [t - 3000, t], including this one.

```
Example: ping(1) -> 1;  ping(100) -> 2;  ping(3001) -> 3;  ping(3002) -> 3
```

Solution (what goes on the board)

```
class RecentCounter:
    def __init__(self):
        self.window = deque()

    def ping(self, t):
        self.window.append(t)
        while self.window[0] < t - 3000:
            self.window.popleft()
        return len(self.window)
```

Reasoning, step by step

1. Only requests from the last 3000 ms matter. Older ones can be forgotten forever, because time only moves forward.
2. Keep the recent request times in a queue: the oldest is at the front, the newest at the back.
3. ping(t): add t at the back.
4. Remove from the front every time that is older than t - 3000.
5. The queue length is the answer. A list with pop(0) would cost O(n) per removal, so use collections.deque.

Complexity

Time	O(1) amortized per ping
Auxiliary space	O(W), at most 3001 entries

Time: each time is appended once and removed once, so across all pings the work is proportional to the number of pings. Space: the queue never holds more than the pings that fit in a 3001 ms window.

Memory estimate

Assumptions. 10,000 pings, one per millisecond (the worst case for window size: 3001 entries). The times are ints created by the caller; the deque holds references to them, so only the deque itself counts.

Part	How it is counted	Estimate
Input	10,000 times as ints: 28n (the caller creates them)	273 KB
Auxiliary	deque of 3001 entries: about 8.3 bytes per entry + 600 bytes overhead	25 KB
Total	input + auxiliary	298 KB

Class 8: Queues and deques
Problem B (harder)

#232 Implement Queue using Stacks

Easy. https://leetcode.com/problems/implement-queue-using-stacks/

Implement a first-in, first-out queue using only two stacks (Python lists used with append and pop at the end). Provide push(x), pop(), peek() and empty().

`Example: push(1); push(2); peek() -> 1; pop() -> 1; empty() -> False`

Solution (what goes on the board)

```
class MyQueue:
    def __init__(self):
        self.inbox = []
        self.outbox = []

    def push(self, x):
        self.inbox.append(x)

    def pop(self):
        self.peek()
        return self.outbox.pop()

    def peek(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        return self.outbox[-1]

    def empty(self):
        return not self.inbox and not self.outbox
```

Reasoning, step by step

1. A stack reverses order; two reversals restore it. That is the whole idea.
2. push goes onto the inbox stack.
3. To remove from the front we need the oldest item, which is at the bottom of the inbox. Pour the whole inbox into the outbox: the order flips, so the oldest is now on top.
4. Only pour when the outbox is empty; otherwise the order of items would be wrong.
5. Each item is moved at most once from inbox to outbox, so the cost per operation is O(1) amortized, even though one pop can cost O(n).

Complexity

Time	O(1) amortized per operation
Auxiliary space	O(n)

Time: a single pop can move n items, but each item moves at most once in its lifetime, so n operations cost O(n) in total. Space: all n items are stored in the two lists.

Memory estimate

Assumptions. n = 100,000 pushes (far more than the LeetCode limit of 100 calls, to show growth). The items are ints made by the caller; the lists hold only pointers. Counted at the moment all n items sit in the inbox.

Part	How it is counted	Estimate
Input	100,000 items as ints: 28n	2.7 MB
Auxiliary	one list of n pointers (about 12% growth room), 8 bytes each	879 KB
Total	input + auxiliary	3.5 MB

Class 9: Monotonic stack and deque
Problem A (easier)

#496 Next Greater Element I

Easy. https://leetcode.com/problems/next-greater-element-i/

nums1 is a subset of nums2; all values are distinct. For each x in nums1, find x in nums2 and return the first value to the right of it in nums2 that is greater than x, or -1 if there is none. Return the answers in the order of nums1.

`Example: nums1 = [4, 1, 2], nums2 = [1, 3, 4, 2]  ->  [-1, 3, -1]`

Solution (what goes on the board)

```
def nextGreaterElement(nums1, nums2):
    greater = {}
    stack = []
    for x in nums2:
        while stack and stack[-1] < x:
            greater[stack.pop()] = x
        stack.append(x)
    return [greater.get(x, -1) for x in nums1]
```

Reasoning, step by step

1. Brute force: for each x find its place in nums2 and scan right. That is O(n x m).
2. Scan nums2 once, keeping a stack of values still waiting for a greater value. The stack stays in decreasing order from bottom to top (a monotonic stack).
3. When x arrives, every value on top of the stack that is smaller than x has found its answer: x. Pop each one and record greater[value] = x.
4. Then push x. Values never popped have no greater value to the right (-1).
5. Finally answer the queries for nums1 with dict lookups.

Complexity

Time	O(n + m)
Auxiliary space	O(n + m)

Time: each value of nums2 is pushed once and popped at most once, plus one lookup per element of nums1. Space: the stack and the dict together hold at most n entries, plus the m-element answer list.

Memory estimate

Assumptions. n = m = 1,000 (the LeetCode limit), nums2 a random permutation, nums1 equal to nums2. The stack and dict hold references to ints that already exist in the input, so no new ints are created.

Part	How it is counted	Estimate
Input	2 lists x (56 + 8n + 28n), sharing the same ints so 28n counted once: 2(56 + 8n) + 28n	43 KB
Auxiliary	dict about 40 bytes per entry for up to n entries + stack 8n + answer list 8n	55 KB
Total	input + auxiliary	98 KB

Other approaches

Approach	Time	Space
Brute force: scan right from each x	O(n x m)	O(1) extra

Class 9: Monotonic stack and deque
Problem B (harder)

#739 Daily Temperatures

Medium. https://leetcode.com/problems/daily-temperatures/

temperatures[i] is the temperature on day i. Return a list answer where answer[i] is the number of days you must wait after day i to get a warmer temperature. If there is no future warmer day, answer[i] = 0.

`Example: temperatures = [73, 74, 75, 71, 69, 72, 76, 73]  ->  [1, 1, 4, 2, 1, 1, 0, 0]`

Solution (what goes on the board)

```
def dailyTemperatures(temperatures):
    answer = [0] * len(temperatures)
    stack = []
    for i, t in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < t:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

Reasoning, step by step

1. Brute force: for each day scan forward until a warmer day. That is O(n^2) in the worst case (a long run of equal or falling temperatures).
2. Keep a stack of day indexes whose warmer day has not yet been found. Their temperatures are non-increasing from bottom to top (a monotonic stack).
3. Today (day i, temperature t) is the warmer day for every index on top of the stack with a lower temperature. Pop each one, j, and set answer[j] = i - j.
4. Then push i. Indexes left on the stack at the end keep the default 0.
5. Each index is pushed once and popped at most once.

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: each index is pushed once and popped at most once, so the while loop does at most n pops in total. Space: the stack can hold every index (for example, when all temperatures are equal), plus the answer list.

Memory estimate

Assumptions. n = 100,000 days (the LeetCode limit), all the same temperature (the worst case for stack size: nobody is ever popped). The answer list holds n zeros (the shared int 0). The stack holds n index ints created by enumerate, each a new 28-byte int above 256.

Part	How it is counted	Estimate
Input	56 + 8n (temperatures between 30 and 100 are cached small ints)	781 KB
Auxiliary	answer list 56 + 8n; stack list about 9n plus 28n for the index ints	4.3 MB

Part	How it is counted	Estimate
Total	input + auxiliary	5.1 MB

Other approaches

Approach	Time	Space
Brute force: scan forward from each day	O(n^2)	O(1) extra

Class 10: Linked lists
Problem A (easier)

#206 Reverse Linked List

Easy. https://leetcode.com/problems/reverse-linked-list/

Given the head of a singly linked list, reverse the list and return the new head.

`Example: 1 -> 2 -> 3 -> 4 -> 5  becomes  5 -> 4 -> 3 -> 2 -> 1`

Solution (what goes on the board)

```
    def reverseList(head):
        prev = None
        cur = head
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt
        return prev
```

Reasoning, step by step

1. Walk the list once with two references: cur (the node we are fixing) and prev (the already reversed part, starting empty).
2. Before changing cur.next, save the rest of the list in nxt. If we skip this, we lose the rest of the list.
3. Point cur.next backward at prev.
4. Advance: prev becomes cur, cur becomes nxt.
5. When cur is None, prev is the old tail, which is the new head.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: one pass, constant work per node. Space: three references, no matter how long the list is.

Memory estimate

Assumptions. n = 5,000 nodes (the LeetCode limit). A ListNode object with an int and a pointer takes about 100 bytes in CPython, ints included in that figure (about 50 bytes of object, plus attribute storage).

Part	How it is counted	Estimate
Input	n nodes x about 100 bytes	488 KB
Auxiliary	three references, about 150 bytes	150 bytes
Total	input + auxiliary	488 KB

Class 10: Linked lists
Problem B (harder)

#19 Remove Nth Node From End of List

Medium. https://leetcode.com/problems/remove-nth-node-from-end-of-list/

Given the head of a singly linked list and an integer n, remove the nth node from the end of the list and return the head. (n = 1 means the last node.) n is always valid.

`Example: head = 1 -> 2 -> 3 -> 4 -> 5, n = 2  ->  1 -> 2 -> 3 -> 5`

Solution (what goes on the board)

```
    def removeNthFromEnd(head, n):
        dummy = ListNode(0, head)
        fast = dummy
        slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            fast = fast.next
            slow = slow.next
        slow.next = slow.next.next
        return dummy.next
```

Reasoning, step by step

1. A singly linked list does not know its length. Counting first needs two passes; two pointers need one.
2. Add a dummy node in front of head so that removing the first node needs no special case.
3. Move fast forward n steps. Now fast and slow are n nodes apart.
4. Move both together until fast is at the last node. Then slow is just before the node to delete.
5. Delete it: slow.next = slow.next.next. Return dummy.next, which is the (possibly new) head.

Complexity

Time	O(L) in one pass, L = length of the list
Auxiliary space	O(1)

Time: fast walks the whole list once and slow walks L - n steps. Space: the dummy node and two references.

Memory estimate

Assumptions. L = 30 nodes (the LeetCode limit). A ListNode takes about 100 bytes.

Part	How it is counted	Estimate
Input	L nodes x about 100 bytes	2.9 KB
Auxiliary	one dummy node (about 100 bytes) + two references	250 bytes
Total	input + auxiliary	3.2 KB

Class 11: Recursion
Problem A (easier)

#509 Fibonacci Number

Easy. https://leetcode.com/problems/fibonacci-number/

The Fibonacci numbers start F(0) = 0, F(1) = 1, and every later number is the sum of the two before it: F(n) = F(n - 1) + F(n - 2). Given n (0 <= n <= 30), return F(n).

```
Example: n = 2  ->  1        n = 3  ->  2        n = 4  ->  3
```

Solution (what goes on the board)

```
    def fib(n):
        memo = {0: 0, 1: 1}

        def go(k):
            if k not in memo:
                memo[k] = go(k - 1) + go(k - 2)
            return memo[k]

        return go(n)
```

Reasoning, step by step

1. Recursive idea: F(k) = F(k - 1) + F(k - 2), with base cases F(0) = 0 and F(1) = 1.
2. Naive recursion calls fib(k - 1) and fib(k - 2) separately, and the same values are recomputed over and over. The call tree roughly doubles at every level: O(2^n) time.
3. Fix: remember every answer in a dict (memoization). Each k is computed once and then looked up.
4. The recursion goes n levels deep before the first return, so the call stack holds up to n frames.
5. Time becomes O(n); space is O(n) for the dict plus O(n) for the call stack.

Complexity

Time	O(n) with memo (naive: O(2^n))
Auxiliary space	O(n)

Time: each k from 0 to n is computed once, with O(1) work. Space: the memo has n + 1 entries and the recursion is n deep.

Memory estimate

Assumptions. n = 30 (the LeetCode limit). The memo dict ends with 31 entries holding ints; the call stack is about 30 frames at roughly 150 bytes each. Frames are counted in the estimate but are invisible to tracemalloc.

Part	How it is counted	Estimate
Input	one int: 28 bytes	28 bytes
Auxiliary	dict with 31 entries (about 1.2 KB) + 31 values x 28 bytes + 30 frames x 150 bytes	6.4 KB
Total	input + auxiliary	6.4 KB

Other approaches

Approach	Time	Space
Naive recursion, no memory of answers	O(2^n)	O(n) stack
Loop with two variables	O(n)	O(1)

Class 11: Recursion
Problem B (harder)

#50 Pow(x, n)

Medium. https://leetcode.com/problems/powx-n/

Implement pow(x, n): compute x raised to the integer power n, where x is a float and n can be negative or zero. (n is between -2^31 and 2^31 - 1, so a loop that multiplies x by itself n times is far too slow.)

```
Example: x = 2.0, n = 10  ->  1024.0        x = 2.0, n = -2  ->  0.25
```

Solution (what goes on the board)

```
def myPow(x, n):
    if n < 0:
        return 1 / myPow(x, -n)
    if n == 0:
        return 1.0
    half = myPow(x, n // 2)
    if n % 2 == 0:
        return half * half
    return half * half * x
```

Reasoning, step by step

1. Multiplying x by itself n times is O(n), and n can be two billion.
2. Use x^n = (x^(n/2)) squared when n is even, and x * (x^((n-1)/2)) squared when n is odd.
3. The exponent halves at every call, so we reach the base case n = 0 after about log2(n) calls.
4. Handle a negative exponent once: x^(-n) = 1 / x^n.
5. Compute half = myPow(x, n // 2) once and reuse it; calling myPow twice would undo the saving.

Complexity

Time	O(log n)
Auxiliary space	O(log n)

Time: n is halved every call, so there are about log2(n) calls (at most 32 for a 32-bit n). Space: the recursion is log2(n) frames deep.

Memory estimate

Assumptions. n = 2^31, so the recursion is about 32 frames deep, each about 150 bytes, plus a few floats of 24 bytes each. All of it is stack space.

Part	How it is counted	Estimate
Input	one float and one int: about 52 bytes	52 bytes
Auxiliary	32 frames x 150 bytes + 32 floats x 24 bytes	5.4 KB
Total	input + auxiliary	5.5 KB

Other approaches

Approach	Time	Space
Loop multiplying n times	O(n)	O(1)
Iterative binary exponentiation (look at the bits of n)	O(log n)	O(1)

Class 12: Bit manipulation
Problem A (easier)

#136 Single Number

Easy. https://leetcode.com/problems/single-number/

In a list nums every value appears exactly twice except one value that appears once. Return the value that appears once, using only O(1) extra space.

```
Example: nums = [4, 1, 2, 1, 2]  ->  4
```

Solution (what goes on the board)

```
    def singleNumber(nums):
        result = 0
        for x in nums:
            result ^= x
        return result
```

Reasoning, step by step

1. A hash set or counter solves it in O(n) time but uses O(n) space. The problem asks for O(1) space.
2. XOR (^) has three useful rules: x ^ x = 0, x ^ 0 = x, and the order does not matter.
3. XOR all numbers together. Every value that appears twice cancels to 0.
4. Only the single value is left: result = that value.
5. Check on [4, 1, 2, 1, 2]: 4 ^ 1 ^ 2 ^ 1 ^ 2 = 4 ^ (1 ^ 1) ^ (2 ^ 2) = 4 ^ 0 ^ 0 = 4.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: one pass with one XOR per element. Space: one integer.

Memory estimate

Assumptions. n = 30,000 values (the LeetCode limit), distinct values above 256 appearing in pairs. Each XOR step produces a temporary int of 28 bytes and frees the previous one.

Part	How it is counted	Estimate
Input	56 + 8n + 28 x n/2 (each value appears twice, as the same int object)	645 KB
Auxiliary	result, x and one temporary int, about 100 bytes	100 bytes
Total	input + auxiliary	645 KB

Other approaches

Approach	Time	Space
Set: add on first sight, remove on second	O(n)	O(n)
Dict of counts	O(n)	O(n)
Sort, then check neighbors in pairs	O(n log n)	O(1)

Class 12: Bit manipulation
Problem B (harder)

#260 Single Number III

Medium. https://leetcode.com/problems/single-number-iii/

In a list nums exactly two values appear once and every other value appears exactly twice. Return the two values that appear once (in any order). Use O(n) time and O(1) extra space.

`Example: nums = [1, 2, 1, 3, 2, 5]  ->  [3, 5]`

Solution (what goes on the board)

```
    def singleNumber(nums):
        xor_all = 0
        for x in nums:
            xor_all ^= x
        low = xor_all & -xor_all
        a = 0
        for x in nums:
            if x & low:
                a ^= x
        return [a, xor_all ^ a]
```

Reasoning, step by step

1. Let the two unique numbers be a and b. XOR of everything is a ^ b, because the pairs cancel.
2. a ^ b is not zero (a and b differ), so it has at least one 1 bit. At that bit, a and b differ: one has 0 and the other has 1.
3. low = xor_all & -xor_all keeps only the lowest 1 bit of xor_all (for example 6 = 110 gives 010).
4. Split the numbers into two groups: those with that bit set, and those without. Each pair lands entirely in one group, so a and b land in different groups.
5. XOR the group with the bit set: pairs cancel and a is left. Then b = xor_all ^ a.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: two passes over the list. Space: four integer variables.

Memory estimate

Assumptions. n = 30,000 values (the LeetCode limit), distinct values in pairs plus two singletons. Temporaries are 28-byte ints that are freed immediately.

Part	How it is counted	Estimate
Input	56 + 8n + 28 x n/2 (each value appears twice, as the same int object)	645 KB
Auxiliary	xor_all, low, a, x and a result list of 2: about 250 bytes	250 bytes
Total	input + auxiliary	645 KB

Other approaches

Approach	Time	Space

Class 13: Sorting
Problem A (easier)

#1331 Rank Transform of an Array

Easy. https://leetcode.com/problems/rank-transform-of-an-array/

Given a list arr, replace each element by its rank: the smallest value has rank 1, the next larger distinct value has rank 2, and so on. Equal values share the same rank. Return the list of ranks.

```
Example: arr = [40, 10, 20, 30]  ->  [4, 1, 2, 3]        arr = [37, 12, 28, 9, 100, 56, 80, 5, 12]  ->  [5, 3, 4, 2, 8, 6, 7, 1, 3]
```

Solution (what goes on the board)

```
def arrayRankTransform(arr):
    rank = {}
    for x in sorted(set(arr)):
        rank[x] = len(rank) + 1
    return [rank[x] for x in arr]
```

Reasoning, step by step

1. Ranks depend on order, so sort. Equal values must share a rank, so remove duplicates first with set(arr).
2. Walk through the sorted distinct values: the first gets rank 1, the next 2, and so on. len(rank) + 1 is the next rank.
3. Store value -> rank in a dict.
4. Rebuild the answer in the original order by looking up each element.
5. The sort dominates the time at O(n log n); the dict, the sorted list and the answer all take O(n) space.

Complexity

Time	O(n log n)
Auxiliary space	O(n)

Time: sorting the distinct values is O(d log d) (d <= n); the two loops are O(n). Space: the set, the sorted list, the dict and the answer list are each O(n).

Memory estimate

Assumptions. n = 100,000 distinct ints above 256. At the peak the set is gone, but the sorted list (8 bytes per slot), the dict (about 52 bytes per entry), the 100,000 new rank ints (28 bytes each) and the answer list (8 bytes per slot) are all alive.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	sorted list 9n + dict 52n + rank ints 28n + answer list 9n	9.3 MB
Total	input + auxiliary	12.8 MB

Other approaches

Approach	Time	Space
Sort (value, index) pairs, then walk once	O(n log n)	O(n)

Class 13: Sorting
Problem B (harder)

#912 Sort an Array

Medium. https://leetcode.com/problems/sort-an-array/

Given a list nums, sort it in ascending order and return it. You must not use the built-in sort. The solution should run in O(n log n) time.

```
Example: nums = [5, 2, 3, 1]  ->  [1, 2, 3, 5]        nums = [5, 1, 1, 2, 0, 0]  ->  [0, 0, 1, 1, 2, 5]
```

Solution (what goes on the board)

```
    def sortArray(nums):
        if len(nums) <= 1:
            return nums
        mid = len(nums) // 2
        left = sortArray(nums[:mid])
        right = sortArray(nums[mid:])
        merged = []
        i = 0
        j = 0
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged
```

Reasoning, step by step

1. Merge sort: split the list in half, sort each half by the same method, then merge the two sorted halves.
2. Base case: a list of 0 or 1 elements is already sorted.
3. Merging two sorted lists: keep an index in each, repeatedly take the smaller front value. Using <= keeps equal values in their original order (stable).
4. When one half runs out, append what is left of the other half.
5. Recursion depth is log2(n). At each level the merges together touch n elements, so the total is n log2(n) steps.

Complexity

Time	O(n log n)
Auxiliary space	O(n)

Time: log2(n) levels, each doing O(n) work in merges and slices. Space: the merged lists and slices at any moment total O(n), plus O(log n) stack frames.

Memory estimate

Assumptions. n = 50,000 ints (the LeetCode limit), random order. The numbers are shared by reference, so only list slots count. At the biggest merge the two sorted halves (n slots) and the merged list (n slots) are alive together, with about 12% growth room on lists built by append. The about 16 stack frames are small and ignored.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	1.7 MB
Auxiliary	sorted halves 8n + merged 8n x 1.12	830 KB
Total	input + auxiliary	2.5 MB

Other approaches

Approach	Time	Space
Insertion sort	O(n^2)	O(1)
Quicksort with random pivot	O(n log n) average, O(n^2) worst	O(log n)
Built-in sorted() (Timsort)	O(n log n)	O(n)

Class 14: Binary search
Problem A (easier)

#704 Binary Search

Easy. https://leetcode.com/problems/binary-search/

Given a list nums sorted in ascending order with distinct values, and an integer target, return the index of target, or -1 if it is not in the list. The solution must run in O(log n) time.

```
Example: nums = [-1, 0, 3, 5, 9, 12], target = 9  ->  4        target = 2  ->
-1
```

Solution (what goes on the board)

```
    def search(nums, target):
        lo = 0
        hi = len(nums) - 1
        while lo <= hi:
            mid = (lo + hi) // 2
            if nums[mid] == target:
                return mid
            if nums[mid] < target:
                lo = mid + 1
            else:
                hi = mid - 1
        return -1
```

Reasoning, step by step

1. Invariant: if target is in the list, it is somewhere in nums[lo..hi] (both ends included).
2. Look at the middle value. If it equals target, we are done.
3. If the middle is smaller than target, the target must be to the right: lo = mid + 1. Otherwise it is to the left: hi = mid - 1.
4. Each step throws away at least half of the remaining range, so there are at most about log2(n) + 1 steps.
5. When lo > hi the range is empty, so the target is not present: return -1.

Complexity

Time	O(log n)
Auxiliary space	O(1)

Time: the range halves every step: n, n/2, n/4, ... , 1, so about log2(n) steps. Space: lo, hi and mid.

Memory estimate

Assumptions. n = 10,000 sorted distinct ints above 256 (the LeetCode limit).

Part	How it is counted	Estimate
Input	56 + 8n + 28n	352 KB
Auxiliary	lo, hi, mid, about 100 bytes	100 bytes
Total	input + auxiliary	352 KB

Other approaches

Approach	Time	Space
Linear scan	O(n)	O(1)
bisect_left from the standard library	O(log n)	O(1)

Class 14: Binary search
Problem B (harder)

#34 Find First and Last Position of Element in Sorted Array

Medium. https://leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/

Given a list nums sorted in non-decreasing order (duplicates allowed) and a target, return [first, last], the first and last index at which target appears, or [-1, -1] if it does not appear. Run in O(log n) time.

```
Example: nums = [5, 7, 7, 8, 8, 10], target = 8  ->  [3, 4]        target = 6
->  [-1, -1]
```

Solution (what goes on the board)

```
    def searchRange(nums, target):
        def first_at_least(x):
            lo = 0
            hi = len(nums)
            while lo < hi:
                mid = (lo + hi) // 2
                if nums[mid] < x:
                    lo = mid + 1
                else:
                    hi = mid
            return lo

        start = first_at_least(target)
        if start == len(nums) or nums[start] != target:
            return [-1, -1]
        return [start, first_at_least(target + 1) - 1]
```

Reasoning, step by step

1. A plain binary search finds some copy of target, not necessarily the first or last. Walking left or right from it could be O(n).
2. Write a helper first_at_least(x): the index of the first element that is >= x (or n if none). It is a binary search that does not stop on equality: when nums[mid] >= x it keeps mid (hi = mid).
3. The first position of target is first_at_least(target). If that index is past the end or the value there is not target, return [-1, -1].
4. The last position of target is first_at_least(target + 1) - 1, because target + 1 is the first value strictly larger (integers).
5. Two binary searches: O(log n) time, O(1) space.

Complexity

Time	O(log n)
Auxiliary space	O(1)

Time: two binary searches of about log2(n) steps each. Space: a few indexes and the 2-element result list.

Memory estimate

Assumptions. n = 100,000 sorted ints above 256 (the LeetCode limit), with a block of 1,000 repeated target values. The result list holds two small ints: 56 + 16 bytes plus up to two 28-byte ints.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	result list (72 bytes) + two ints + lo, hi, mid, about 250 bytes	250 bytes
Total	input + auxiliary	3.4 MB

Other approaches

Approach	Time	Space
Linear scan	O(n)	O(1)
bisect_left and bisect_right	O(log n)	O(1)

Class 15: Trees: traversal
Problem A (easier)

#104 Maximum Depth of Binary Tree

Easy. https://leetcode.com/problems/maximum-depth-of-binary-tree/

Given the root of a binary tree, return its maximum depth: the number of nodes along the longest path from the root down to a leaf.

```
Example: root = [3, 9, 20, null, null, 15, 7]  ->  3        root = []  ->  0
```

Solution (what goes on the board)

```
    def maxDepth(root):
        if root is None:
            return 0
        left = maxDepth(root.left)
        right = maxDepth(root.right)
        return 1 + max(left, right)
```

Reasoning, step by step

1. A tree is made of smaller trees: a root plus a left subtree and a right subtree. That suggests recursion.
2. Base case: an empty tree has depth 0.
3. Otherwise the depth is 1 (for this node) plus the larger of the two subtree depths.
4. Every node is visited exactly once, so the time is O(n).
5. Space is the recursion depth, which is the height h of the tree: O(log n) if balanced, O(n) if the tree is a chain.

Complexity

Time	O(n)
Auxiliary space	O(h), h = height of the tree

Time: one call per node. Space: the call stack is as deep as the tree is tall; a balanced tree has h of about log2(n).

Memory estimate

Assumptions. n = 10,000 nodes in a balanced tree (height 14). A TreeNode costs about 100 bytes. A skewed tree of 10,000 nodes would need 10,000 frames, which is more than the default Python recursion limit of 1,000.

Part	How it is counted	Estimate
Input	n nodes x about 100 bytes	977 KB
Auxiliary	14 frames x 150 bytes	2.1 KB
Total	input + auxiliary	979 KB

Other approaches

Approach	Time	Space
Iterative level-order (BFS), count the levels	O(n)	O(width)

Class 15: Trees: traversal
Problem B (harder)

#102 Binary Tree Level Order Traversal

Medium. https://leetcode.com/problems/binary-tree-level-order-traversal/

Given the root of a binary tree, return its level order traversal: a list of lists, where list i holds the values of all nodes at depth i, from left to right.

`Example: root = [3, 9, 20, null, null, 15, 7]  ->  [[3], [9, 20], [15, 7]]`

Solution (what goes on the board)

```
    def levelOrder(root):
        if root is None:
            return []
        result = []
        queue = deque([root])
        while queue:
            level = []
            for _ in range(len(queue)):
                node = queue.popleft()
                level.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            result.append(level)
        return result
```

Reasoning, step by step

1. Recursion goes deep first. We want one level at a time, which is breadth-first search, and a queue gives it to us: first in, first out.
2. Start with the root in a deque.
3. At the start of each pass, len(queue) is exactly the number of nodes on the current level. Process that many nodes.
4. For each node: pop it from the front, record its value, and add its children to the back (they form the next level).
5. When the queue is empty every level has been recorded.

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: each node is added to the queue once and removed once. Space: the queue holds at most one level, which can be about n / 2 nodes in a full tree; the result stores all n values.

Memory estimate

Assumptions. n = 10,000 nodes in a complete tree (14 levels, the last level about 1,700 to 5,000 nodes). The values are existing ints, so the result lists hold pointers: 8 bytes per value plus a 56-byte header per level. The queue at its widest holds about n / 2 pointers at 8.3 bytes each plus its 600-byte base.

Part	How it is counted	Estimate
Input	n nodes x about 100 bytes	977 KB
Auxiliary	result lists 8n + 14 x 56, queue (n / 2) x 8.3 + 600	120 KB
Total	input + auxiliary	1.1 MB

Other approaches

Approach	Time	Space
Recursive DFS passing the depth, appending to result[depth]	O(n)	O(h) + result

Class 16: Binary search trees
Problem A (easier)

#700 Search in a Binary Search Tree

Easy. https://leetcode.com/problems/search-in-a-binary-search-tree/

Given the root of a binary search tree and a value val, return the node whose value equals val, or None if there is no such node. (The result is the subtree rooted at that node.)

```
Example: root = [4, 2, 7, 1, 3], val = 2  ->  the subtree [2, 1, 3]
val = 5  ->  None
```

Solution (what goes on the board)

```
    def searchBST(root, val):
        node = root
        while node is not None and node.val != val:
            if val < node.val:
                node = node.left
            else:
                node = node.right
        return node
```

Reasoning, step by step

1. In a binary search tree, everything left of a node is smaller and everything right is larger.
2. So one comparison tells us which whole side to ignore, like binary search.
3. Start at the root. If val is smaller than the node, go left; otherwise go right.
4. Stop when we find val or fall off the tree (node is None). Return the node.
5. We follow one root-to-leaf path: O(h) time and O(1) space with a loop (a recursive version would use O(h) stack).

Complexity

Time	O(h)
Auxiliary space	O(1)

Time: at most one step per level of the tree, so O(h): about log2(n) if balanced, n if the tree is a chain.
Space: a single reference.

Memory estimate

Assumptions. n = 5,000 nodes (the LeetCode limit) in a balanced BST. A TreeNode costs about 100 bytes.

Part	How it is counted	Estimate
Input	n nodes x about 100 bytes	488 KB
Auxiliary	one reference, about 100 bytes	100 bytes
Total	input + auxiliary	488 KB

Class 16: Binary search trees
Problem B (harder)

#98 Validate Binary Search Tree

Medium. https://leetcode.com/problems/validate-binary-search-tree/

Given the root of a binary tree, return True if it is a valid binary search tree: for every node, all values in its left subtree are strictly smaller and all values in its right subtree are strictly larger (and both subtrees are valid).

```
Example: root = [2, 1, 3]  ->  True        root = [5, 1, 4, null, null, 3, 6]
->  False   (3 is in the right subtree of 5 but is smaller than 5)
```

Solution (what goes on the board)

```
    def isValidBST(root):
        def ok(node, low, high):
            if node is None:
                return True
            if not (low < node.val < high):
                return False
            return ok(node.left, low, node.val) and ok(node.right,
    node.val, high)

        return ok(root, float('-inf'), float('inf'))
```

Reasoning, step by step

1. A common mistake: only compare a node with its direct children. In [5, 1, 4, null, null, 3, 6] each child looks fine, but 3 sits to the right of 5 and is smaller than 5.
2. Every node must lie inside a range (low, high) set by its ancestors. The root may be anything: (-infinity, +infinity).
3. Going left, the current value becomes the new upper limit: ok(node.left, low, node.val).
4. Going right, the current value becomes the new lower limit: ok(node.right, node.val, high).
5. If any node falls outside its range, return False. Every node is visited at most once.

Complexity

Time	O(n)
Auxiliary space	O(h)

Time: each node is checked once. Space: the recursion is as deep as the tree is tall.

Memory estimate

Assumptions. n = 10,000 nodes in a balanced valid BST (height 14). Frames cost about 150 bytes; each call holds four values.

Part	How it is counted	Estimate
Input	n nodes x about 100 bytes	977 KB
Auxiliary	14 frames x 150 bytes + two floats (24 bytes each)	2.1 KB
Total	input + auxiliary	979 KB

Class 17: Heaps and priority queues
Problem A (easier)

#703 Kth Largest Element in a Stream

Easy. https://leetcode.com/problems/kth-largest-element-in-a-stream/

Design a class KthLargest. It is created with an integer k and an initial list nums. add(val) adds val to the stream and returns the kth largest element of everything seen so far (the kth largest in sorted order, counting duplicates).

```
Example: KthLargest(3, [4, 5, 8, 2]);  add(3) -> 4;  add(5) -> 5;  add(10) -> 5;  add(9) -> 8;  add(4) -> 8
```

Solution (what goes on the board)

```
import heapq


class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = []
        for x in nums:
            self.add(x)

    def add(self, val):
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

Reasoning, step by step

1. Sorting the whole stream on every add costs O(n log n) per call.
2. We only need the k largest values. Keep them in a min-heap of size k: the smallest of the k largest sits at the top, heap[0].
3. That top value is exactly the kth largest of everything seen.
4. add(val): push val. If the heap now has more than k values, pop the smallest. The heap again holds the k largest.
5. Push and pop are O(log k), and peeking at heap[0] is O(1).

Complexity

Time	O(log k) per add
Auxiliary space	O(k)

Time: one push and at most one pop on a heap of at most k + 1 elements. Space: the heap never holds more than k values.

Memory estimate

Assumptions. k = 10,000 (the LeetCode limit) with a stream of 100,000 distinct ints above 256 created by the caller. The heap list holds pointers to the ints that survive; those ints belong to the stream, so they are not counted as new. A list of k pointers with about 12% growth room.

Part	How it is counted	Estimate
Input	stream of n ints: 28n	2.7 MB
Auxiliary	heap list: 56 + 9k with k = 10,000	88 KB
Total	input + auxiliary	2.8 MB

Class 17: Heaps and priority queues
Problem B (harder)

#347 Top K Frequent Elements

Medium. https://leetcode.com/problems/top-k-frequent-elements/

Given a list nums and an integer k, return the k most frequent values (in any order). The answer is guaranteed to be unique. Better than O(n log n) is expected.

`Example: nums = [1, 1, 1, 2, 2, 3], k = 2  ->  [1, 2]`

Solution (what goes on the board)

```
import heapq


def topKFrequent(nums, k):
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    heap = []
    for x, c in counts.items():
        heapq.heappush(heap, (c, x))
        if len(heap) > k:
            heapq.heappop(heap)
    return [x for c, x in heap]
```

Reasoning, step by step

1. Step 1: count every value with a dict: value -> frequency. That is O(n) time.
2. Step 2: we want the k largest counts. Sorting all d distinct values costs O(d log d).
3. Use a min-heap of size k holding (count, value) pairs. The smallest count is at the top.
4. For each pair: push it; if the heap has more than k pairs, pop the smallest. The heap keeps the k most frequent.
5. At the end the heap holds the answer: read the values out.

Complexity

Time	O(n + d log k), d = number of distinct values
Auxiliary space	O(d + k)

Time: counting is O(n); each of the d distinct values costs one push and at most one pop on a heap of size k + 1, so O(d log k). Space: the dict holds d entries and the heap holds k.

Memory estimate

Assumptions. n = 100,000 values, all distinct (d = n), and k = 100. The dict (about 52 bytes per entry) is the main cost; its count values are the cached small int 1. The heap holds 100 tuples at 56 bytes each plus its list, and the result list holds 100 pointers.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.4 MB
Auxiliary	dict 52d + heap 100 x 56 + 56 + 8 x 100 + result 56 + 8 x 100	5.0 MB

Part	How it is counted	Estimate
Total	input + auxiliary	8.4 MB

Other approaches

Approach	Time	Space
Sort the (count, value) pairs	O(n + d log d)	O(d)
Bucket sort by frequency	O(n)	O(n)

Class 18: Graphs and depth-first search
Problem A (easier)

#733 Flood Fill

Easy. https://leetcode.com/problems/flood-fill/

image is a grid of integers (a picture of pixel colors). Starting from the pixel (sr, sc), change its color and the color of every pixel connected to it through up, down, left or right moves, as long as those pixels have the same starting color, to a new color. Return the changed image.

```
Example: image = [[1, 1, 1], [1, 1, 0], [1, 0, 1]], sr = 1, sc = 1, color = 2
->  [[2, 2, 2], [2, 2, 0], [2, 0, 1]]
```

Solution (what goes on the board)

```
    def floodFill(image, sr, sc, color):
        old = image[sr][sc]
        if old == color:
            return image
        rows = len(image)
        cols = len(image[0])
        image[sr][sc] = color
        stack = [(sr, sc)]
        while stack:
            r, c = stack.pop()
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c -
    1)):
                if 0 <= nr < rows and 0 <= nc < cols and image[nr][nc] ==
    old:
                    image[nr][nc] = color
                    stack.append((nr, nc))
        return image
```

Reasoning, step by step

1. Think of the grid as a graph: each cell is a vertex and its four neighbors are its edges. We want every vertex reachable through cells of the old color: depth-first search.
2. If old == color, nothing would change and the search could loop forever, so return at once.
3. Recolor a cell at the moment we push it. The new color means "already visited", so no separate visited set is needed.
4. Pop a cell and look at its four neighbors. Push each one that is inside the grid and still has the old color.
5. An explicit stack avoids Python recursion limits on large regions. Each cell is pushed at most once.

Complexity

Time	O(m x n)
Auxiliary space	O(m x n)

Time: each cell is pushed and popped at most once and checks four neighbors. Space: in the worst case the stack holds a large share of the cells.

Memory estimate

Assumptions. A 50 x 50 grid (the LeetCode limit, n = 2,500 cells), all one color (the worst case). All values are small cached ints, so the grid costs only list slots: 50 lists of 50 pointers. Each stack entry is a tuple (56 bytes) plus a pointer slot; row and column ints below 257 are cached.

Part	How it is counted	Estimate
Input	50 lists x (56 + 8 x 50) + outer list (56 + 8 x 50)	23 KB
Auxiliary	at most about half of the cells on the stack, each (56 + 8 bytes)	79 KB
Total	input + auxiliary	101 KB

Other approaches

Approach	Time	Space
Recursive DFS	O(m x n)	O(m x n) stack in the worst case
BFS with a deque	O(m x n)	O(m + n) to O(m x n)

Class 18: Graphs and depth-first search
Problem B (harder)

#200 Number of Islands

Medium. https://leetcode.com/problems/number-of-islands/

Given a grid of the characters "1" (land) and "0" (water), count the islands. An island is a group of land cells connected horizontally or vertically. The edges of the grid are surrounded by water.

```
Example: grid = [["1","1","0","0","0"], ["1","1","0","0","0"],
["0","0","1","0","0"], ["0","0","0","1","1"]]  ->  3
```

Solution (what goes on the board)

```
    def numIslands(grid):
        rows = len(grid)
        cols = len(grid[0])
        count = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1':
                    count += 1
                    grid[r][c] = '0'
                    stack = [(r, c)]
                    while stack:
                        cr, cc = stack.pop()
                        for nr, nc in ((cr + 1, cc), (cr - 1, cc), (cr, cc
    + 1), (cr, cc - 1)):
                            if 0 <= nr < rows and 0 <= nc < cols and
    grid[nr][nc] == '1':
                                grid[nr][nc] = '0'
                                stack.append((nr, nc))
        return count
```

Reasoning, step by step

1. Scan every cell. The first time we meet a land cell we have found a new island: count it.
2. Then erase the whole island so we never count it again: depth-first search from that cell, turning every connected "1" into "0" (the sunk cell doubles as the visited mark).
3. Mark a cell when we push it, not when we pop it, so no cell is pushed twice.
4. The outer scan visits each cell once; the searches together visit each land cell once.
5. Overall every cell is handled a constant number of times.

Complexity

Time	O(m x n)
Auxiliary space	O(m x n)

Time: m x n cells scanned, and each land cell is pushed and popped once. Space: in the worst case (one big island) the stack can hold a large share of the cells.

Memory estimate

Assumptions. A 300 x 300 grid (the LeetCode limit, n = 90,000 cells) that is all land, so one island. The grid holds 1-character strings, which are shared, so only list slots count. Each stack entry is a tuple of 56

bytes plus an 8-byte slot, and the row and column ints above 256 are new objects; at its peak the stack holds about half of the cells.

Part	How it is counted	Estimate
Input	300 lists x (56 + 8 x 300) + outer list	722 KB
Auxiliary	stack holds about half of the cells at its peak, each a 56-byte tuple + 8-byte slot + new ints above 256: n / 2 x 72	3.1 MB
Total	input + auxiliary	3.8 MB

Other approaches

Approach	Time	Space
Recursive DFS	O(m x n)	O(m x n) stack
BFS with a deque	O(m x n)	O(min(m, n)) queue
Union-find (class 22)	O(m x n x alpha)	O(m x n)

Class 19: Breadth-first search
Problem A (easier)

#1971 Find if Path Exists in Graph

Easy. https://leetcode.com/problems/find-if-path-exists-in-graph/

There are n vertices numbered 0 to n - 1 and a list of undirected edges. Return True if there is a path from source to destination, and False otherwise.

```
Example: n = 3, edges = [[0, 1], [1, 2], [2, 0]], source = 0, destination = 2
->  True        n = 6, edges = [[0, 1], [0, 2], [3, 5], [5, 4], [4, 3]],
source = 0, destination = 5  ->  False
```

Solution (what goes on the board)

```
    def validPath(n, edges, source, destination):
        graph = {i: [] for i in range(n)}
        for a, b in edges:
            graph[a].append(b)
            graph[b].append(a)
        seen = {source}
        queue = deque([source])
        while queue:
            node = queue.popleft()
            if node == destination:
                return True
            for nxt in graph[node]:
                if nxt not in seen:
                    seen.add(nxt)
                    queue.append(nxt)
        return False
```

Reasoning, step by step

1. A list of edges is awkward to search. Convert it to an adjacency list: a dict from each vertex to the list of its neighbors. Edges are undirected, so add both directions.
2. Breadth-first search from source: a queue holds vertices to explore, a set remembers vertices already reached.
3. Pop a vertex. If it is the destination, there is a path.
4. Otherwise add every unseen neighbor to the set and to the queue.
5. If the queue empties without reaching destination, the destination is in a different component: False.

Complexity

Time	O(V + E)
Auxiliary space	O(V + E)

Time: building the graph is O(V + E); BFS pops each vertex once and looks at each edge twice. Space: the adjacency list holds V lists and 2E entries; the set and queue hold at most V vertices.

Memory estimate

Assumptions. n = 100,000 vertices in a single long path (edges [i, i + 1]); the LeetCode limit is 200,000. The graph dict costs about 52 bytes per entry, a key int of 28 bytes (created by range) and a one- or two-

element list of 88 bytes per vertex. The set adds about 42 bytes per vertex at this size; the queue stays tiny on a path.

Part	How it is counted	Estimate
Input	edges: n pairs, each (56 + 2 x 8) list plus ints: about n x (72 + 2 x 28) + 56 + 8n	13.0 MB
Auxiliary	graph: n x (52 + 28 + 88); seen set: 42n; queue: small	20.0 MB
Total	input + auxiliary	33.0 MB

Other approaches

Approach	Time	Space
DFS with a stack	O(V + E)	O(V + E)
Union-find (class 22)	O((V + E) alpha)	O(V)

Class 19: Breadth-first search
Problem B (harder)

#994 Rotting Oranges

Medium. https://leetcode.com/problems/rotting-oranges/

A grid holds 0 (empty), 1 (fresh orange) and 2 (rotten orange). Every minute, each fresh orange that touches a rotten orange (up, down, left, right) becomes rotten. Return the minimum number of minutes until no fresh orange remains, or -1 if that is impossible.

```
Example: grid = [[2, 1, 1], [1, 1, 0], [0, 1, 1]]  ->  4        grid = [[2,
1, 1], [0, 1, 1], [1, 0, 1]]  ->  -1        grid = [[0, 2]]  ->  0
```

Solution (what goes on the board)

```
    def orangesRotting(grid):
        rows = len(grid)
        cols = len(grid[0])
        queue = deque()
        fresh = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 2:
                    queue.append((r, c))
                elif grid[r][c] == 1:
                    fresh += 1
        minutes = 0
        while queue and fresh:
            for _ in range(len(queue)):
                r, c = queue.popleft()
                for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c -
    1)):
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc]
    == 1:
                        grid[nr][nc] = 2
                        fresh -= 1
                        queue.append((nr, nc))
            minutes += 1
        return minutes if fresh == 0 else -1
```

Reasoning, step by step

1. All rotten oranges spread at the same time. That is a breadth-first search that starts with every rotten orange in the queue at once (multi-source BFS).
2. One BFS level equals one minute. Process exactly len(queue) cells per minute, as in class 15 level order.
3. Count the fresh oranges first. Each time one rots, subtract 1 and add it to the queue.
4. Stop when the queue is empty or there are no fresh oranges left. The number of levels processed is the answer.
5. If any fresh orange is left, it was never reached: return -1.

Complexity

Time	O(m x n)

Auxiliary space	O(m x n)

Time: every cell is scanned once, and each orange enters the queue at most once. Space: the queue can hold many cells at once, up to O(m x n).

Memory estimate

Assumptions. A 10 x 10 grid (the LeetCode limit, n = 100 cells), a single rotten orange in a corner, all other cells fresh. The queue is a deque (about 760 bytes for one block) holding (row, column) tuples of 56 bytes each; at most about 10 cells are on the same BFS level.

Part	How it is counted	Estimate
Input	10 lists x (56 + 80) + outer list	1.5 KB
Auxiliary	deque 760 + about 10 live tuples x 64 + the tuples created along the way (freed)	1.9 KB
Total	input + auxiliary	3.4 KB

Other approaches

Approach	Time	Space
Simulate each minute by rescanning the whole grid	O((m x n)^2)	O(1)

Class 20: Topological sort
Problem A (easier)

#207 Course Schedule

Medium. https://leetcode.com/problems/course-schedule/

There are numCourses courses numbered 0 to numCourses - 1. prerequisites[i] = [a, b] means you must take course b before course a. Return True if it is possible to finish all courses (that is, there is no cycle of prerequisites).

```
Example: numCourses = 2, prerequisites = [[1, 0]]  ->  True        numCourses = 2, prerequisites = [[1, 0], [0, 1]]  ->  False
```

Solution (what goes on the board)

```
    def canFinish(numCourses, prerequisites):
        graph = [[] for _ in range(numCourses)]
        indegree = [0] * numCourses
        for course, pre in prerequisites:
            graph[pre].append(course)
            indegree[course] += 1
        queue = deque(i for i in range(numCourses) if indegree[i] == 0)
        done = 0
        while queue:
            node = queue.popleft()
            done += 1
            for nxt in graph[node]:
                indegree[nxt] -= 1
                if indegree[nxt] == 0:
                    queue.append(nxt)
        return done == numCourses
```

Reasoning, step by step

1. Draw an edge from each prerequisite to the course that needs it. We can finish everything exactly when this directed graph has no cycle.
2. The indegree of a course is the number of prerequisites it still waits for. A course with indegree 0 can be taken right now.
3. Queue every course with indegree 0. Take one (count it) and remove its edges: reduce the indegree of every course that depended on it.
4. Any course whose indegree reaches 0 joins the queue (Kahn's algorithm).
5. If we took all numCourses courses, there is no cycle. Courses on a cycle never reach indegree 0, so done stays smaller.

Complexity

Time	O(V + E)
Auxiliary space	O(V + E)

Time: building the graph is O(V + E); each vertex enters the queue once and each edge is relaxed once. Space: the adjacency lists hold E entries, and the indegree list and queue hold V.

Memory estimate

Assumptions. V = 2,000 courses (the LeetCode limit) in one long chain of prerequisites, E = 1,999. Each adjacency list holds one number and costs 88 bytes; the indegree list holds small cached ints; the queue holds one course at a time, in a deque of about 760 bytes. The index ints from range are new 28-byte ints above 256.

Part	How it is counted	Estimate
Input	prerequisites: E pairs of (56 + 16) bytes + ints	266 KB
Auxiliary	graph: 56 + 8V + V x 88; indegree 56 + 8V; queue 760; ints about 28V	259 KB
Total	input + auxiliary	524 KB

Other approaches

Approach	Time	Space
DFS with three colors (unvisited, in progress, done)	O(V + E)	O(V + E)

Class 20: Topological sort
Problem B (harder)

#210 Course Schedule II

Medium. https://leetcode.com/problems/course-schedule-ii/

The same setup as Course Schedule: prerequisites[i] = [a, b] means take course b before course a. Return one valid order in which you can take all the courses. If it is impossible (there is a cycle), return an empty list.

```
Example: numCourses = 4, prerequisites = [[1, 0], [2, 0], [3, 1], [3, 2]]  ->
[0, 1, 2, 3]   (or [0, 2, 1, 3])
```

Solution (what goes on the board)

```
    def findOrder(numCourses, prerequisites):
        graph = [[] for _ in range(numCourses)]
        indegree = [0] * numCourses
        for course, pre in prerequisites:
            graph[pre].append(course)
            indegree[course] += 1
        queue = deque(i for i in range(numCourses) if indegree[i] == 0)
        order = []
        while queue:
            node = queue.popleft()
            order.append(node)
            for nxt in graph[node]:
                indegree[nxt] -= 1
                if indegree[nxt] == 0:
                    queue.append(nxt)
        if len(order) == numCourses:
            return order
        return []
```

Reasoning, step by step

1. Same graph and the same algorithm as Course Schedule (Kahn's algorithm). The only change: keep the courses in the order we take them.
2. A course is appended to order at the moment it is popped; all its prerequisites were popped earlier, because it only entered the queue when its indegree reached 0.
3. So the order list is a valid topological order.
4. If order has fewer than numCourses entries, some courses sit on a cycle: return [].
5. Several answers are correct; any valid order is accepted.

Complexity

Time	O(V + E)
Auxiliary space	O(V + E)

Time: same as Course Schedule. Space: the same lists plus the order list of V entries.

Memory estimate

Assumptions. V = 2,000 courses in a chain (E = 1,999). The structures are the same as Course Schedule, plus the order list holding V pointers to ints (9 bytes per slot with growth room).

Part	How it is counted	Estimate
Input	prerequisites: E pairs of (56 + 16) bytes + ints	266 KB
Auxiliary	graph 56 + 8V + 88V; indegree 56 + 8V; queue 760; order 56 + 9V; ints 28V	276 KB
Total	input + auxiliary	542 KB

Other approaches

Approach	Time	Space
DFS: record vertices in reverse finishing order	O(V + E)	O(V + E)

Class 21: Shortest paths: Dijkstra
Problem A (easier)

#743 Network Delay Time

Medium. https://leetcode.com/problems/network-delay-time/

A network has n nodes labeled 1 to n. times[i] = [u, v, w] is a directed edge: a signal sent from u reaches v after w time units. A signal is sent from node k. Return the time until all n nodes have received it, or -1 if some node can never receive it.

`Example: times = [[2, 1, 1], [2, 3, 1], [3, 4, 1]], n = 4, k = 2  ->  2`

Solution (what goes on the board)

```
import heapq


def networkDelayTime(times, n, k):
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))
    dist = {}
    heap = [(0, k)]
    while heap:
        d, node = heapq.heappop(heap)
        if node in dist:
            continue
        dist[node] = d
        for nxt, w in graph[node]:
            if nxt not in dist:
                heapq.heappush(heap, (d + w, nxt))
    if len(dist) < n:
        return -1
    return max(dist.values())
```

Reasoning, step by step

1. The time for the signal to reach a node is the shortest-path distance from k to that node, with edge weights as times. The answer is the largest of those distances (the last node to be reached).
2. Dijkstra: keep a min-heap of (distance so far, node). The smallest entry is the next node whose distance is final.
3. Pop (d, node). If node is already finished (in dist), this entry is stale: skip it. Otherwise dist[node] = d.
4. For each neighbor not yet finished, push (d + w, neighbor). Several entries for one node may be on the heap; only the cheapest is used.
5. If fewer than n nodes were finished, some node is unreachable: -1. Otherwise return the largest distance.

Complexity

Time	O(E log E), written O((V + E) log V)
Auxiliary space	O(V + E)

Time: each edge pushes at most one heap entry and each heap operation is O(log E); building the graph is O(V + E). Space: the adjacency lists hold E edges, the heap holds at most E entries, and dist holds V entries.

Memory estimate

Assumptions. V = 100 nodes and E = 6,000 edges (the LeetCode limits), random edges with weights 1 to 100. Each edge is stored once as a 2-tuple in a graph list (56 + 8 bytes) and the heap can hold up to one 2-tuple (56 + 8 bytes) per edge; distances above 256 are new 28-byte ints. Assume the heap peaks at about half the edges.

Part	How it is counted	Estimate
Input	times: E triples of (64 + 8) bytes	469 KB
Auxiliary	graph 6,000 x 64 + heap about 3,000 x 64 + dist dict 100 x 52 + ints	590 KB
Total	input + auxiliary	1.0 MB

Other approaches

Approach	Time	Space
Bellman-Ford: relax every edge V - 1 times	O(V x E)	O(V)
BFS (only if all weights are equal)	O(V + E)	O(V)

Class 21: Shortest paths: Dijkstra
Problem B (harder)

#1631 Path With Minimum Effort

Medium. https://leetcode.com/problems/path-with-minimum-effort/

heights is a grid of integers. You start at the top-left cell and want to reach the bottom-right cell, moving up, down, left or right. The effort of a route is the largest absolute height difference between two consecutive cells on it. Return the minimum effort over all routes.

`Example: heights = [[1, 2, 2], [3, 8, 2], [5, 3, 5]]  ->  2`

Solution (what goes on the board)

```
import heapq


def minimumEffortPath(heights):
    rows = len(heights)
    cols = len(heights[0])
    best = [[float('inf')] * cols for _ in range(rows)]
    best[0][0] = 0
    heap = [(0, 0, 0)]
    while heap:
        effort, r, c = heapq.heappop(heap)
        if r == rows - 1 and c == cols - 1:
            return effort
        if effort > best[r][c]:
            continue
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c -
1)):
            if 0 <= nr < rows and 0 <= nc < cols:
                new = max(effort, abs(heights[nr][nc] - heights[r]
[c]))
                if new < best[nr][nc]:
                    best[nr][nc] = new
                    heapq.heappush(heap, (new, nr, nc))
```

Reasoning, step by step

1. Each cell is a vertex; each move to a neighbor is an edge. The "distance" is not a sum: it is the largest step so far along the route. That still works for Dijkstra, because adding a step can never make the effort smaller.
2. best[r][c] is the smallest effort found so far to reach (r, c); start with infinity everywhere and 0 at the start.
3. Pop the cell with the smallest effort from a min-heap. The first time the goal is popped, its effort is the answer.
4. For each neighbor the effort of going through the current cell is max(effort, |height difference|). If that is better than best for the neighbor, update it and push.
5. Skip stale heap entries whose effort is larger than best for their cell.

Complexity

Time	O(m n log(m n))

Auxiliary space	O(m n)

Time: each cell can be pushed a few times (once per improvement, at most 4), and each push or pop costs O(log(m n)). Space: the best table has m n entries and the heap holds up to a few entries per cell.

Memory estimate

Assumptions. A 100 x 100 grid (the LeetCode limit, n = 10,000 cells) that is flat (every height 1) except the goal cell, so the search explores every cell at effort 0 before reaching the goal. The best table is 100 lists of 100 pointers to the cached int 0 or the shared float infinity. The heap holds only the search frontier, about 200 entries of one 3-tuple (64 bytes + 8-byte slot) each. For adversarial heights a cell can be pushed up to 4 times, so the heap can reach about 4n entries (about 3 MB).

Part	How it is counted	Estimate
Input	100 lists x (56 + 800) + outer list	84 KB
Auxiliary	best table 101 x 856 + frontier of about 200 entries x 72	98 KB
Total	input + auxiliary	183 KB

Other approaches

Approach	Time	Space
Binary search on the answer + BFS	O(m n log H)	O(m n)
Sort edges + union-find (class 22)	O(m n log(m n))	O(m n)

Class 22: Union-find
Problem A (easier)

#547 Number of Provinces

Medium. https://leetcode.com/problems/number-of-provinces/

There are n cities. isConnected is an n x n matrix: isConnected[i][j] = 1 if cities i and j are directly connected, else 0. A province is a group of cities connected directly or indirectly. Return the number of provinces.

```
Example: isConnected = [[1, 1, 0], [1, 1, 0], [0, 0, 1]]  ->  2
isConnected = [[1, 0, 0], [0, 1, 0], [0, 0, 1]]  ->  3
```

Solution (what goes on the board)

```
    def findCircleNum(isConnected):
        n = len(isConnected)
        parent = list(range(n))

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        provinces = n
        for i in range(n):
            for j in range(i + 1, n):
                if isConnected[i][j] == 1:
                    root_i = find(i)
                    root_j = find(j)
                    if root_i != root_j:
                        parent[root_i] = root_j
                        provinces -= 1
        return provinces
```

Reasoning, step by step

1. Start with every city in its own group: parent[i] = i, so the root of each group is the city itself. There are n provinces.
2. find(x) follows parent links up to the root. Path halving (parent[x] = parent[parent[x]]) shortens the chain as we climb, so later finds are faster.
3. For every connected pair (i, j): find both roots. If they differ, the groups are separate, so join them (point one root at the other) and reduce the province count by one.
4. If the roots are equal the cities are already in the same province: do nothing.
5. The number of provinces left after all pairs is the answer.

Complexity

Time	O(n^2) (the matrix scan dominates; each find is almost constant time)
Auxiliary space	O(n)

Time: reading the upper half of the n x n matrix is about n^2 / 2 steps, and each union-find operation is almost O(1) amortized. Space: the parent list holds n entries.

Memory estimate

Assumptions. n = 200 cities (the LeetCode limit), nothing connected except each city to itself (the worst case for scanning). The ints 0 to 199 are cached small ints, so the parent list costs only its pointer slots. The matrix is 200 lists of 200 pointers to the cached ints 0 and 1.

Part	How it is counted	Estimate
Input	200 lists x (56 + 1,600) + outer list	325 KB
Auxiliary	parent list 56 + 8n + a few ints	1.8 KB
Total	input + auxiliary	327 KB

Other approaches

Approach	Time	Space
DFS or BFS over the matrix	O(n^2)	O(n)

Class 22: Union-find
Problem B (harder)

#684 Redundant Connection

Medium. https://leetcode.com/problems/redundant-connection/

A tree with n nodes (labeled 1 to n) had one extra edge added, so the input is a list of n edges that contains exactly one cycle. Return the edge that can be removed so the rest is a tree. If several edges work, return the one that appears last in the input.

```
Example: edges = [[1, 2], [1, 3], [2, 3]]  ->  [2, 3]        edges = [[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]  ->  [1, 4]
```

Solution (what goes on the board)

```
    def findRedundantConnection(edges):
        parent = list(range(len(edges) + 1))

        def find(x):
            while parent[x] != x:
                parent[x] = parent[parent[x]]
                x = parent[x]
            return x

        for a, b in edges:
            root_a = find(a)
            root_b = find(b)
            if root_a == root_b:
                return [a, b]
            parent[root_a] = root_b
```

Reasoning, step by step

1. Add the edges one by one, keeping each connected group as a union-find tree.
2. An edge (a, b) whose endpoints are already in the same group closes a cycle: that edge is redundant.
3. Otherwise the edge joins two groups: point one root at the other.
4. The first edge that closes a cycle is the last one in the input that can be removed, because the input contains exactly one cycle and we stop at the edge that completes it.
5. Without union-find, a DFS from a to b before adding each edge would cost O(n) per edge and O(n^2) in total.

Complexity

Time	O(n x alpha(n)), essentially O(n)
Auxiliary space	O(n)

Time: n edges, each needing two near-constant finds. Space: the parent list of n + 1 entries.

Memory estimate

Assumptions. n = 1,000 edges (the LeetCode limit) forming a path 1-2-...-n with the cycle-closing edge [1, n] last. The parent list holds n + 1 pointers; the ints above 256 are new 28-byte objects created by list(range(...)).

Part	How it is counted	Estimate
Input	n edges x (56 + 16) bytes + outer list + ints about 28 x 2n	133 KB
Auxiliary	parent list 56 + 8(n + 1) + 28n ints	35 KB
Total	input + auxiliary	168 KB

Other approaches

Approach	Time	Space
DFS: before adding each edge, check if a and b are already connected	O(n^2)	O(n)

Class 23: Tries
Problem A (easier)

#208 Implement Trie (Prefix Tree)

Medium. https://leetcode.com/problems/implement-trie-prefix-tree/

Implement a Trie: insert(word) stores a word; search(word) returns True if the exact word was inserted; startsWith(prefix) returns True if any inserted word begins with prefix. All strings use lowercase letters.

```
Example: insert("apple"); search("apple") -> True; search("app") -> False;
startsWith("app") -> True; insert("app"); search("app") -> True
```

Solution (what goes on the board)

```
class Trie:
    def __init__(self):
        self.root = {}

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node['$'] = True

    def search(self, word):
        node = self.walk(word)
        return node is not None and '$' in node

    def startsWith(self, prefix):
        return self.walk(prefix) is not None

    def walk(self, text):
        node = self.root
        for ch in text:
            if ch not in node:
                return None
            node = node[ch]
        return node
```

Reasoning, step by step

1. A trie is a tree where each edge is a letter and a path from the root spells a prefix. Words that share a prefix share the same path.
2. Each node is a dict: letter -> child node. The special key "$" marks that a word ends at this node.
3. insert: walk down the word, creating a child dict for any missing letter, then set "$" on the last node.
4. walk(text) follows a string from the root and returns the node it ends on, or None if the path breaks. search needs that node to have "$"; startsWith only needs the node to exist.
5. Each operation costs O(L) for a word of length L, no matter how many words are stored. A set of words would find exact words in O(L) too, but could not answer prefix questions without scanning everything.

Complexity

Time	O(L) per operation
Auxiliary space	O(total characters stored)

Time: each operation touches one node per letter. Space: at most one node (a dict) per stored character, fewer when words share prefixes.

Memory estimate

Assumptions. 10,000 random lowercase words of 8 letters each (80,000 letters). A node is a dict: one key costs about 184 bytes, several keys cost about 270 bytes. The number of nodes is about the sum over depths d = 1..8 of min(26^d, 10,000) (the layers fill up after depth 3), roughly 53,000 nodes at about 200 bytes each. The words are created by the caller.

Part	How it is counted	Estimate
Input	10,000 words x (41 + 8) + list	557 KB
Auxiliary	about 53,000 nodes x 200 bytes	11.6 MB
Total	input + auxiliary	12.1 MB

Class 23: Tries
Problem B (harder)

#212 Word Search II

Hard. https://leetcode.com/problems/word-search-ii/

Given an m x n board of letters and a list of words, return all the words that can be formed by a path of adjacent cells (horizontal or vertical). A cell may be used only once per word. Each word is distinct.

```
Example: board = [["o","a","a","n"], ["e","t","a","e"], ["i","h","k","r"], ["i","f","l","v"]], words = ["oath","pea","eat","rain"]  ->  ["eat", "oath"]
```

Solution (what goes on the board)

```
    def findWords(board, words):
        root = {}
        for word in words:
            node = root
            for ch in word:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node['$'] = word
        rows = len(board)
        cols = len(board[0])
        found = []

        def dfs(r, c, parent):
            ch = board[r][c]
            node = parent[ch]
            if '$' in node:
                found.append(node.pop('$'))
            board[r][c] = '#'
            for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 
    1)):
                if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] in 
    node:
                    dfs(nr, nc, node)
            board[r][c] = ch
            if not node:
                parent.pop(ch)

        for r in range(rows):
            for c in range(cols):
                if board[r][c] in root:
                    dfs(r, c, root)
        return found
```

Reasoning, step by step

1. Class 24 idea in advance: Word Search I runs a backtracking search from every cell for one word. With many words that repeats the same work for each word.
2. Put all the words into one trie. A path on the board is only worth following while its letters form a prefix that is in the trie. A letter that has no child in the trie prunes the whole branch.

3. dfs(r, c, parent) steps onto a cell whose letter is a child of parent. If that trie node ends a word, record it (and delete the marker so the word is not found twice).
4. Mark the cell as used with "#", search the four neighbors, then restore the letter (backtrack).
5. Pruning: when a trie node has no children left, remove it from its parent so later searches never walk into dead ends.

Complexity

Time	O(M x 4 x 3^(L - 1)), M = cells, L = longest word
Auxiliary space	O(total letters in words) for the trie + O(L) recursion

Time: each of the M start cells explores at most 4 x 3^(L - 1) paths of length up to L (three choices after the first step, because we cannot go back). The trie prunes most of them. Space: the trie holds up to one node per letter of every word; the recursion is at most L deep.

Memory estimate

Assumptions. 10,000 random 8-letter words (as for Trie) stored in the trie before the search starts, on a 12 x 12 random board. The trie is the peak: about 53,000 nodes at about 200 bytes each. The words themselves are created by the caller; each word end stores a reference to the existing word.

Part	How it is counted	Estimate
Input	10,000 words x 49 + list + board 12 x (56 + 96)	559 KB
Auxiliary	about 53,000 trie nodes x 200 bytes	11.6 MB
Total	input + auxiliary	12.1 MB

Other approaches

Approach	Time	Space
Run Word Search I for every word	O(W x M x 3^L)	O(L)

Class 24: Backtracking
Problem A (easier)

#78 Subsets

Medium. https://leetcode.com/problems/subsets/

Given a list nums of distinct integers, return all possible subsets (the power set), in any order. The result must not contain duplicate subsets.

```
Example: nums = [1, 2, 3]  ->  [[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]
```

Solution (what goes on the board)

```
    def subsets(nums):
        result = []
        path = []

        def backtrack(start):
            result.append(path[:])
            for i in range(start, len(nums)):
                path.append(nums[i])
                backtrack(i + 1)
                path.pop()

        backtrack(0)
        return result
```

Reasoning, step by step

1. A subset is a sequence of choices: for each number, in the order of the list, take it or leave it. A set of n numbers therefore has 2^n subsets.
2. path holds the subset being built. Every path we ever reach is itself a valid subset, so copy it into the result at every call (path[:] makes a copy; without the copy all entries would be the same list).
3. The loop picks the next number to add. Starting at index start (not 0) means we only add numbers that come later, so each subset is built in exactly one order and none repeats.
4. Choose: path.append. Explore: backtrack(i + 1). Undo: path.pop().
5. The search tree has 2^n nodes, and copying a path costs up to n.

Complexity

Time	O(n x 2^n)
Auxiliary space	O(n) for the recursion and path (the output itself is O(n x 2^n))

Time: there are 2^n subsets, and copying each into the result costs up to n. Space: the recursion is at most n deep; the output holds n / 2 x 2^n numbers in total.

Memory estimate

Assumptions. n = 10 (the LeetCode limit): 1,024 subsets with 5,120 entries in total. Each subset is a list of 56 bytes plus 8 bytes per element (copies are exact-size); the numbers are shared. The result list holds 1,024 pointers (with growth room). We count the output as auxiliary space.

Part	How it is counted	Estimate
Input	list of 10 ints	416 bytes
Auxiliary	1,024 lists x 56 + 8 x 5,120 + result list 9 x 1,024 + path	105 KB
Total	input + auxiliary	106 KB

Other approaches

Approach	Time	Space
Bitmask: for mask in range(2^n), take the numbers whose bit is set	O(n x 2^n)	O(1) extra besides output

Class 24: Backtracking
Problem B (harder)

#51 N-Queens

Hard. https://leetcode.com/problems/n-queens/

Place n queens on an n x n chessboard so that no two queens attack each other (no two share a row, column or diagonal). Return every distinct solution, each as a list of n strings with "Q" for a queen and "." for an empty square.

```
Example: n = 4  ->  [[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...",
"...Q", ".Q.."]]
```

Solution (what goes on the board)

```
    def solveNQueens(n):
        cols = set()
        diag1 = set()
        diag2 = set()
        placed = []
        result = []

        def place(r):
            if r == n:
                result.append(['.' * c + 'Q' + '.' * (n - c - 1) for c in
    placed])
                return
            for c in range(n):
                if c in cols or (r - c) in diag1 or (r + c) in diag2:
                    continue
                cols.add(c)
                diag1.add(r - c)
                diag2.add(r + c)
                placed.append(c)
                place(r + 1)
                placed.pop()
                cols.remove(c)
                diag1.remove(r - c)
                diag2.remove(r + c)

        place(0)
        return result
```

Reasoning, step by step

1. Exactly one queen goes in each row, so the choice at step r is only the column for row r. That removes the need to look at rows.
2. Queens clash in a column (same c), on a "\" diagonal (same r - c) or on a "/" diagonal (same r + c). Keep three sets of what is taken, so each check is O(1).
3. place(r): try every column c. If c, r - c or r + c is taken, skip it (prune). Otherwise add them to the sets, record c, and recurse to row r + 1.
4. Backtrack: remove the column from placed and the three entries from the sets so the next column starts clean.
5. When r == n all rows have a queen: build the board strings from placed and save them.

Complexity

Time	O(n!) in the worst case (the pruning makes it far smaller in practice)
Auxiliary space	O(n) for the search; O(n^2) per solution for the output

Time: row 0 has n choices, row 1 at most n - 1 safe columns, and so on, so at most n! placements are explored. Space: the recursion is n deep and the sets hold at most n entries each.

Memory estimate

Assumptions. n = 8: 92 solutions, each a list of 8 strings of 8 characters. A string costs 41 + 8 = 49 bytes, a solution list 56 + 64 bytes. The three sets hold up to 8 small ints each (about 730 bytes at that fill). The output is counted as auxiliary space; the 8-frame stack is invisible to tracemalloc and about 8 x 200 bytes.

Part	How it is counted	Estimate
Input	one int: 28 bytes	28 bytes
Auxiliary	92 solutions x (120 + 8 x 49) + result list 9 x 92 + 3 sets x 730 + 8 frames x 200	51 KB
Total	input + auxiliary	51 KB

Other approaches

Approach	Time	Space
Check the whole board for attacks after each placement	O(n x n!)	O(n^2)

Class 25: Greedy algorithms
Problem A (easier)

#55 Jump Game

Medium. https://leetcode.com/problems/jump-game/

nums[i] is the maximum jump length from position i. Starting at index 0, return True if you can reach the last index, and False otherwise.

```
Example: nums = [2, 3, 1, 1, 4]  ->  True        nums = [3, 2, 1, 0, 4]  ->
False
```

Solution (what goes on the board)

```
    def canJump(nums):
        farthest = 0
        for i, jump in enumerate(nums):
            if i > farthest:
                return False
            farthest = max(farthest, i + jump)
        return True
```

Reasoning, step by step

1. Trying every jump length from every position (recursion or backtracking) is exponential; with memoization it is still O(n^2).
2. Greedy idea: we do not care how we get to a position, only how far we can get. Keep farthest, the farthest index reachable so far.
3. Scan left to right. If position i is beyond farthest, no earlier jump can reach it: return False.
4. Otherwise position i is reachable, and it extends the reach to i + nums[i]: farthest = max(farthest, i + jump).
5. If the scan finishes, every index was reachable, including the last one.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: one pass. Space: farthest, i and jump.

Memory estimate

Assumptions. n = 10,000 values (the LeetCode limit), all 1 (reach moves one step at a time). The small ints are cached, so the input costs only pointer slots.

Part	How it is counted	Estimate
Input	56 + 8n	78 KB
Auxiliary	farthest, i, jump and the enumerate object, about 200 bytes	200 bytes
Total	input + auxiliary	78 KB

Other approaches

Approach	Time	Space
Backtracking with memoization (DP)	O(n^2)	O(n)

Class 25: Greedy algorithms
Problem B (harder)

#135 Candy

Hard. https://leetcode.com/problems/candy/

n children stand in a line, and ratings[i] is the rating of child i. Give each child at least one candy. A child with a higher rating than a neighbor must get more candies than that neighbor. Return the minimum total number of candies.

```
Example: ratings = [1, 0, 2]  ->  5   (candies 2, 1, 2)        ratings = [1, 2, 2]  ->  4   (candies 1, 2, 1)
```

Solution (what goes on the board)

```
    def candy(ratings):
        n = len(ratings)
        candies = [1] * n
        for i in range(1, n):
            if ratings[i] > ratings[i - 1]:
                candies[i] = candies[i - 1] + 1
        for i in range(n - 2, -1, -1):
            if ratings[i] > ratings[i + 1]:
                candies[i] = max(candies[i], candies[i + 1] + 1)
        return sum(candies)
```

Reasoning, step by step

1. There are two rules, one for each neighbor. Trying to satisfy both at once is awkward, so satisfy them one at a time.
2. Start everyone at 1 candy.
3. Left-to-right pass: if a child has a higher rating than the left neighbor, give one more than that neighbor. Now every child is fine relative to the left.
4. Right-to-left pass: if a child has a higher rating than the right neighbor, they need at least one more than that neighbor; keep the larger of the old value and the new requirement, so the left rule still holds.
5. Each child gets the minimum that satisfies both neighbors, so the sum is the minimum total (greedy: giving less to any child would break a rule).

Complexity

Time	O(n)
Auxiliary space	O(n)

Time: two passes and a sum. Space: the candies list of n entries (an O(1)-space solution exists by counting up and down slopes).

Memory estimate

Assumptions. n = 20,000 children (the LeetCode limit), ratings strictly increasing (the worst case for the size of the candy counts: 1, 2, ..., n). Counts above 256 are new 28-byte ints.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	703 KB

Part	How it is counted	Estimate
Auxiliary	candies list 56 + 8n, plus a new int for each count above 256	696 KB
Total	input + auxiliary	1.4 MB

Other approaches

Approach	Time	Space
Two arrays: left[i] and right[i], take the max	O(n)	O(n)
Slope counting	O(n)	O(1)

Class 26: Intervals and sweep line
Problem A (easier)

#56 Merge Intervals

Medium. https://leetcode.com/problems/merge-intervals/

Given a list of intervals [start, end], merge all overlapping intervals and return the non-overlapping intervals that cover the same ranges. Intervals that touch at an endpoint (like [1, 4] and [4, 5]) overlap.

`Example: intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]  ->  [[1, 6], [8, 10], [15, 18]]`

Solution (what goes on the board)

```
    def merge(intervals):
        merged = []
        for start, end in sorted(intervals):
            if merged and start <= merged[-1][1]:
                merged[-1][1] = max(merged[-1][1], end)
            else:
                merged.append([start, end])
        return merged
```

Reasoning, step by step

1. If the intervals are in random order, any interval might overlap any other, giving O(n^2) comparisons. Sort by start first.
2. After sorting, an interval can only overlap the last merged interval, because all earlier merged intervals end before it starts.
3. For each interval, compare its start with the end of the last merged interval. If start <= that end, they overlap: extend the end to max(old end, new end).
4. Otherwise start a new merged interval.
5. The sort costs O(n log n); the sweep is one pass.

Complexity

Time	O(n log n)
Auxiliary space	O(n)

Time: sorting dominates; the sweep is O(n). Space: sorted() makes a list of n pointers, and the result holds up to n new interval lists.

Memory estimate

Assumptions. n = 10,000 disjoint intervals (the LeetCode limit and the worst case: nothing merges). The sorted copy holds n pointers to the existing interval lists; the result holds n new 2-element lists of 72 bytes each (the ints are shared with the input).

Part	How it is counted	Estimate
Input	n intervals x (72 + 2 x 28) + outer list	1.3 MB
Auxiliary	sorted copy 56 + 8n + result n x 72 + result list 9n	869 KB
Total	input + auxiliary	2.1 MB

Other approaches

Approach	Time	Space
Compare every pair and merge repeatedly	O(n^2)	O(n)

Class 26: Intervals and sweep line
Problem B (harder)

#218 The Skyline Problem

Hard. https://leetcode.com/problems/the-skyline-problem/

Each building is [left, right, height]. Return the skyline: the list of key points [x, height] where the height of the outline changes, sorted by x. The last point has height 0. Consecutive key points must have different heights.

```
Example: buildings = [[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]  ->  [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]
```

Solution (what goes on the board)

```
import heapq


def getSkyline(buildings):
    events = []
    for left, right, height in buildings:
        events.append((left, -height, right))
        events.append((right, 0, 0))
    events.sort()
    result = []
    heap = [(0, float('inf'))]
    for x, neg_height, right in events:
        while heap[0][1] <= x:
            heapq.heappop(heap)
        if neg_height != 0:
            heapq.heappush(heap, (neg_height, right))
        top = -heap[0][0]
        if not result or result[-1][1] != top:
            result.append([x, top])
    return result
```

Reasoning, step by step

1. The outline can only change at a building edge (a left or a right x). Make an event for each: (x, -height, right) for a start, and (x, 0, 0) for an end.
2. Sort the events by x. At equal x, the key (x, -height, ...) puts taller starts first, and puts end events (-height = 0) after every start. This ordering avoids dropping to a lower height and back at the same x.
3. Sweep left to right with a max-heap of the buildings that are "open". Python has a min-heap, so store (-height, right): the tallest building is at heap[0].
4. At each event first discard buildings from the top whose right edge is at or before x (they have ended; lazy deletion: ended buildings buried deeper in the heap are removed only when they reach the top). A sentinel (0, infinity) represents the ground and is never removed.
5. After handling the event, the current skyline height is the top of the heap. If it differs from the last recorded height, add [x, height].

Complexity

Time	O(n log n)

Auxiliary space	O(n)

Time: sorting 2n events is O(n log n); each building is pushed and popped from the heap once, O(log n) each. Space: the event list has 2n entries and the heap at most n + 1.

Memory estimate

Assumptions. n = 10,000 buildings (the LeetCode limit). The events list holds 2n 3-tuples (64 bytes plus an 8-byte slot, with growth room), each with a new negative-height int (28 bytes). The heap peaks near n 2-tuples (56 + 8 bytes) when all buildings overlap. Here each building is taller than the last and starts one unit later, so every start adds a key point: the result holds n key points of 72 bytes plus a new 28-byte int each.

Part	How it is counted	Estimate
Input	n buildings x (56 + 24 + 3 x 28) + outer list	1.6 MB
Auxiliary	events 2n x (64 + 9) + negative ints 28n + heap n x 65 + sort buffer 8n + result n x (72 + 28 + 9)	3.4 MB
Total	input + auxiliary	5.0 MB

Other approaches

Approach	Time	Space
Compute the maximum height at every integer x (small coordinates only)	O(n x W)	O(W)
Divide and conquer: merge two skylines	O(n log n)	O(n)

Class 27: Dynamic programming: 1D
Problem A (easier)

#198 House Robber

Medium. https://leetcode.com/problems/house-robber/

nums[i] is the money in house i along a street. You cannot rob two adjacent houses. Return the maximum amount you can rob.

```
Example: nums = [1, 2, 3, 1]  ->  4   (rob houses 0 and 2)        nums = [2,
7, 9, 3, 1]  ->  12   (rob houses 0, 2 and 4)
```

Solution (what goes on the board)

```
def rob(nums):
    prev = 0
    cur = 0
    for x in nums:
        prev, cur = cur, max(cur, prev + x)
    return cur
```

Reasoning, step by step

1. State: best[i] = the most money from the first i houses. For house i there are two choices: skip it (best[i - 1]) or rob it (best[i - 2] + nums[i], since house i - 1 is then off limits).
2. Recurrence: best[i] = max(best[i - 1], best[i - 2] + nums[i]). Base cases: best[0] = 0 (no houses) and best[1] = nums[0].
3. A table of n + 1 numbers works (O(n) space), but each entry only looks back two places.
4. So keep just two variables: prev = best[i - 2] and cur = best[i - 1].
5. At each house compute the new value, then shift: prev, cur = cur, new. The answer is cur.

Complexity

Time	O(n)
Auxiliary space	O(1)

Time: one pass, constant work per house. Space: two variables (a full table would be O(n)).

Memory estimate

Assumptions. n = 100 houses (the LeetCode limit), values up to 400. The sums are new 28-byte ints (values above 256).

Part	How it is counted	Estimate
Input	56 + 8n + 28n	3.6 KB
Auxiliary	prev, cur, x and temporaries, about 150 bytes	150 bytes
Total	input + auxiliary	3.7 KB

Other approaches

Approach	Time	Space
Plain recursion without memory	O(2^n)	O(n)

Approach	Time	Space
DP table best[0..n]	O(n)	O(n)

Class 27: Dynamic programming: 1D
Problem B (harder)

#322 Coin Change

Medium. https://leetcode.com/problems/coin-change/

coins is a list of coin values (unlimited supply of each) and amount is a target. Return the fewest coins that make exactly amount, or -1 if it cannot be made.

```
Example: coins = [1, 2, 5], amount = 11  ->  3   (5 + 5 + 1)        coins =
[2], amount = 3  ->  -1
```

Solution (what goes on the board)

```
    def coinChange(coins, amount):
        INF = amount + 1
        dp = [0] + [INF] * amount
        for a in range(1, amount + 1):
            for coin in coins:
                if coin <= a and dp[a - coin] + 1 < dp[a]:
                    dp[a] = dp[a - coin] + 1
        if dp[amount] == INF:
            return -1
        return dp[amount]
```

Reasoning, step by step

1. Greedy fails: with coins [1, 3, 4] and amount 6, taking 4 first gives 4 + 1 + 1 (3 coins), but 3 + 3 uses 2.
2. State: dp[a] = the fewest coins to make amount a. Base: dp[0] = 0.
3. Recurrence: the last coin used is some coin c, so dp[a] = 1 + min(dp[a - c]) over every coin c <= a.
4. Fill a from 1 up to amount, so dp[a - c] is already known. INF = amount + 1 stands for "impossible" (it is larger than any real answer).
5. If dp[amount] is still INF, no combination works: return -1.

Complexity

Time	O(amount x number of coins)
Auxiliary space	O(amount)

Time: amount + 1 states, each trying every coin. Space: the dp list of amount + 1 entries.

Memory estimate

Assumptions. amount = 10,000 (the LeetCode limit) with coins [1, 2, 5]. The dp list holds pointers; its values are new ints for counts above 256 (about 87% of the entries here, since the counts reach 2,000).

Part	How it is counted	Estimate
Input	3 coins: list of 3	164 bytes
Auxiliary	dp list 56 + 8n + 28 x 0.87n	316 KB
Total	input + auxiliary	316 KB

Other approaches

Approach	Time	Space
Recursion without memory	O(coins^amount)	O(amount)
BFS over amounts	O(amount x coins)	O(amount)
Greedy (incorrect in general)	-	-

Class 28: Dynamic programming: sequences
Problem A (easier)

#300 Longest Increasing Subsequence

Medium. https://leetcode.com/problems/longest-increasing-subsequence/

Given a list nums, return the length of the longest strictly increasing subsequence (the numbers need not be next to each other, but must keep their order).

`Example: nums = [10, 9, 2, 5, 3, 7, 101, 18]  ->  4   (for example 2, 3, 7, 101)`

Solution (what goes on the board)

```
def lengthOfLIS(nums):
    n = len(nums)
    dp = [1] * n
    for i in range(n):
        for j in range(i):
            if nums[j] < nums[i] and dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
    return max(dp)
```

Reasoning, step by step

1. State: dp[i] = the length of the longest increasing subsequence that ENDS at position i. (If we only recorded "best in the first i numbers" we would not know whether the next number can extend it.)
2. Every position alone is a subsequence of length 1, so start dp at 1.
3. Recurrence: dp[i] = 1 + max(dp[j]) over earlier positions j with nums[j] < nums[i]. Try every earlier j.
4. The answer is the best ending anywhere: max(dp).
5. n states, each looking at up to n earlier values: O(n^2) time, O(n) space. There is an O(n log n) solution using binary search on a list of tails.

Complexity

Time	O(n^2)
Auxiliary space	O(n)

Time: for each i the inner loop runs i times, which sums to n(n - 1)/2 steps. Space: the dp list of n entries.

Memory estimate

Assumptions. n = 2,500 values (the LeetCode limit), strictly increasing (the worst case for dp values: 1, 2, ..., n). Counts above 256 are new 28-byte ints.

Part	How it is counted	Estimate
Input	56 + 8n + 28n	88 KB
Auxiliary	dp list 56 + 8n + 28 x (n - 256) new ints	81 KB
Total	input + auxiliary	169 KB

Other approaches

Approach	Time	Space
Tails list + binary search (bisect_left)	O(n log n)	O(n)
Plain recursion	O(2^n)	O(n)

Class 28: Dynamic programming: sequences
Problem B (harder)

#1143 Longest Common Subsequence

Medium. https://leetcode.com/problems/longest-common-subsequence/

Given two strings text1 and text2, return the length of their longest common subsequence: the longest sequence of characters that appears in both in the same order (not necessarily next to each other). Return 0 if there is none.

```
Example: text1 = "abcde", text2 = "ace"  ->  3   ("ace")        text1 =
"abc", text2 = "def"  ->  0
```

Solution (what goes on the board)

```
    def longestCommonSubsequence(text1, text2):
        m = len(text1)
        n = len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + 1
                else:
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
        return dp[m][n]
```

Reasoning, step by step

1. State: dp[i][j] = the LCS length of the first i characters of text1 and the first j characters of text2. Row 0 and column 0 are 0 (an empty string has nothing in common).
2. If the last characters text1[i - 1] and text2[j - 1] are equal, they extend the best common subsequence of the shorter prefixes: dp[i - 1][j - 1] + 1.
3. If they differ, at least one of them is not used: take the better of dropping the last character of text1 (dp[i - 1][j]) or of text2 (dp[i][j - 1]).
4. Fill row by row; each cell needs only the cell above, to the left, and diagonally above-left, which are already filled.
5. The answer is the bottom-right cell. Keeping only the previous row reduces space to O(n).

Complexity

Time	O(m x n)
Auxiliary space	O(m x n)

Time: m x n cells, each in constant time. Space: the (m + 1) x (n + 1) table.

Memory estimate

Assumptions. m = n = 1,000 random lowercase strings (the LeetCode limit). The table is 1,001 lists of 1,001 pointers: 1,001 x (56 + 8,008) bytes. Counts above 256 would be new 28-byte ints; for random strings of 1,000 letters the LCS is about 400, so the few matching cells with counts above 256 add about 0.4 MB.

Part	How it is counted	Estimate
Input	2 strings x (41 + n)	2.0 KB
Auxiliary	table (n + 1) x (56 + 8(n + 1)) + new ints for matches above 256, about 0.4 MB	8.1 MB
Total	input + auxiliary	8.1 MB

Other approaches

Approach	Time	Space
Plain recursion without memory	O(2^(m + n))	O(m + n)
Two rows only	O(m x n)	O(min(m, n))

Class 29: Dynamic programming: 2D tables
Problem A (easier)

#72 Edit Distance

Medium. https://leetcode.com/problems/edit-distance/

Given two words word1 and word2, return the minimum number of single-character operations (insert, delete or replace a character) needed to turn word1 into word2.

`Example: word1 = "horse", word2 = "ros"  ->  3   (horse -> rorse -> rose -> ros)`

Solution (what goes on the board)

```
    def minDistance(word1, word2):
        m = len(word1)
        n = len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]
        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i -
    1][j - 1])
        return dp[m][n]
```

Reasoning, step by step

1. State: dp[i][j] = the fewest operations to turn the first i characters of word1 into the first j characters of word2.
2. Base cases: turning i characters into nothing takes i deletions (dp[i][0] = i); turning nothing into j characters takes j insertions (dp[0][j] = j).
3. If the last characters match, no operation is needed for them: dp[i][j] = dp[i - 1][j - 1].
4. If they differ, try the three operations and take the cheapest plus one: delete from word1 (dp[i - 1][j]), insert into word1 (dp[i][j - 1]) or replace (dp[i - 1][j - 1]).
5. The answer is dp[m][n]. Fill three cells by hand on "ab" -> "b" to check the recurrence.

Complexity

Time	O(m x n)
Auxiliary space	O(m x n)

Time: m x n cells, each in constant time. Space: the (m + 1) x (n + 1) table (two rows are enough if you do not need the actual edits).

Memory estimate

Assumptions. m = n = 500 random lowercase strings (the LeetCode limit). The table is 501 lists of 501 pointers. Values above 256 are new 28-byte ints; for random 500-letter strings the distance is about 400 and about two thirds of the 250,000 cells hold a new int above 256.

Part	How it is counted	Estimate
Input	2 strings x (41 + n)	1.1 KB
Auxiliary	table (n + 1) x (56 + 8(n + 1)) + 28 bytes for a new int in about 0.65 n^2 cells	6.3 MB
Total	input + auxiliary	6.3 MB

Other approaches

Approach	Time	Space
Plain recursion without memory	O(3^(m + n))	O(m + n)
Two rows only	O(m x n)	O(min(m, n))

Class 29: Dynamic programming: 2D tables
Problem B (harder)

#10 Regular Expression Matching

Hard. https://leetcode.com/problems/regular-expression-matching/

Implement regular expression matching with "." (matches any single character) and "*" (matches zero or more copies of the character just before it). Return True if the whole string s matches the whole pattern p.

```
Example: s = "aa", p = "a"  ->  False        s = "aa", p = "a*"  ->  True
s = "ab", p = ".*"  ->  True
```

Solution (what goes on the board)

```
    def isMatch(s, p):
        m = len(s)
        n = len(p)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True
        for j in range(2, n + 1):
            if p[j - 1] == '*':
                dp[0][j] = dp[0][j - 2]
        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if p[j - 1] == '*':
                    dp[i][j] = dp[i][j - 2]
                    if p[j - 2] == s[i - 1] or p[j - 2] == '.':
                        dp[i][j] = dp[i][j] or dp[i - 1][j]
                elif p[j - 1] == s[i - 1] or p[j - 1] == '.':
                    dp[i][j] = dp[i - 1][j - 1]
        return dp[m][n]
```

Reasoning, step by step

1. State: dp[i][j] = True if the first i characters of s match the first j characters of p. dp[0][0] = True (empty matches empty).
2. Row 0: an empty string can match a pattern only if every starred item can be used zero times: dp[0][j] = dp[0][j - 2] when p[j - 1] is "*".
3. Ordinary character or ".": dp[i][j] is True when the last characters agree (p[j - 1] equals s[i - 1] or is ".") and dp[i - 1][j - 1] is True.
4. Star at p[j - 1] acts on the letter before it, p[j - 2]. Zero copies: dp[i][j - 2]. One more copy (only if p[j - 2] matches s[i - 1] or is "."): dp[i - 1][j], because the star can still absorb more.
5. The answer is dp[m][n]. Trace "aa" against "a*" in a 3 x 3 table to check the three rules.

Complexity

Time	O(m x n)
Auxiliary space	O(m x n)

Time: (m + 1) x (n + 1) cells, each in constant time. Space: the same table of booleans (the booleans True and False are shared objects).

Memory estimate

Assumptions. m = n = 20 (the LeetCode limit). The table is 21 lists of 21 pointers to the shared objects True and False.

Part	How it is counted	Estimate
Input	2 strings x (41 + 20)	122 bytes
Auxiliary	table 21 x (56 + 8 x 21) + outer list + a few ints	4.9 KB
Total	input + auxiliary	5.0 KB

Other approaches

Approach	Time	Space
Recursion without memory	exponential	O(m + n)
Top-down recursion with memo	O(m x n)	O(m x n)

Class 30: Capstone: choosing the pattern
Problem A (easier)

#127 Word Ladder

Hard. https://leetcode.com/problems/word-ladder/

beginWord and endWord are words of the same length, and wordList is a list of allowed words. A move changes exactly one letter, and every word you reach must be in wordList. Return the number of words in the shortest sequence from beginWord to endWord (counting both ends), or 0 if no sequence exists.

`Example: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]  ->  5   (hit -> hot -> dot -> dog -> cog)`

Solution (what goes on the board)

```
    def ladderLength(beginWord, endWord, wordList):
        words = set(wordList)
        if endWord not in words:
            return 0
        queue = deque([(beginWord, 1)])
        seen = {beginWord}
        while queue:
            word, steps = queue.popleft()
            if word == endWord:
                return steps
            for i in range(len(word)):
                for ch in 'abcdefghijklmnopqrstuvwxyz':
                    nxt = word[:i] + ch + word[i + 1:]
                    if nxt in words and nxt not in seen:
                        seen.add(nxt)
                        queue.append((nxt, steps + 1))
        return 0
```

Reasoning, step by step

1. Model it as a graph: each word is a vertex, and two words are joined by an edge when they differ in one letter. We want the shortest path from beginWord to endWord measured in edges, plus one for counting words: breadth-first search.
2. We do not build the graph. The neighbors of a word are generated on demand: change each position to each of 26 letters and keep results that are in the word set (a set gives O(1) lookup).
3. BFS from beginWord with a queue of (word, number of words so far). The first time endWord is popped, steps is the answer.
4. Mark a word as seen when it is added to the queue, so no word is queued twice.
5. If the queue empties, endWord is unreachable: return 0.

Complexity

Time	O(N x L^2 x 26), N = words, L = word length
Auxiliary space	O(N x L)

Time: each of N words is dequeued once; for each we try L positions x 26 letters, and building each candidate string costs O(L). Space: the word set, the seen set and the queue hold up to N words of L letters each.

Memory estimate

Assumptions. N = 3,125 words (all 5-letter words over the letters a to e), L = 5, beginWord "aaaaa" and endWord "eeeee". The word set and the seen set take about 42 bytes per word each (table sizes grow in steps). Each seen word is a new string made by the search (41 + 5 bytes); the queue holds 2-tuples (56 + 8 bytes) with step counts (small cached ints). The peak is about when the queue has emptied half of the graph.

Part	How it is counted	Estimate
Input	3,125 words x (41 + 5) + list	165 KB
Auxiliary	word set 42N + seen set 42N + new strings 46N + queue tuples about N x 36	507 KB
Total	input + auxiliary	671 KB

Other approaches

Approach	Time	Space
Bidirectional BFS (search from both ends)	about O(N x L^2 x 26), usually much less	O(N x L)
Wildcard buckets: "h*t" -> words	O(N x L^2)	O(N x L^2)

Class 30: Capstone: choosing the pattern
Problem B (harder)

#295 Find Median from Data Stream

Hard. https://leetcode.com/problems/find-median-from-data-stream/

Design a MedianFinder. addNum(num) adds an integer from a data stream; findMedian() returns the median of all numbers so far: the middle value if the count is odd, or the mean of the two middle values if it is even.

```
Example: addNum(1); addNum(2); findMedian() -> 1.5; addNum(3); findMedian()
-> 2.0
```

Solution (what goes on the board)

```
import heapq


class MedianFinder:
    def __init__(self):
        self.small = []
        self.large = []

    def addNum(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2
```

Reasoning, step by step

1. Keeping the list sorted costs O(n) per insert (shifting); sorting at every query costs O(n log n). We only ever need the middle.
2. Split the numbers into two halves: small holds the smaller half, large holds the larger half. Then the median is at the boundary between them.
3. small must give its LARGEST value quickly: a max-heap. Python has only a min-heap, so store negatives in small. large is a normal min-heap giving its smallest value.
4. Invariant: every value in small is <= every value in large, and len(small) is len(large) or len(large) + 1.
5. addNum: push into small, move small's largest over to large (this keeps the order invariant), and if large is now bigger, move its smallest back. findMedian: the top of small if the count is odd, else the mean of the two tops.

Complexity

Time	addNum O(log n); findMedian O(1)
Auxiliary space	O(n)

Time: addNum does at most three heap operations on heaps of size at most n / 2 + 1; findMedian reads two tops. Space: every number is stored once in one of the heaps.

Memory estimate

Assumptions. n = 50,000 numbers added (the LeetCode limit). Both heaps hold new ints made by negating the input (each 28 bytes), and the two lists have 8-byte slots with about 12% growth room. The caller's own ints are not retained.

Part	How it is counted	Estimate
Input	stream of n ints: 28n	1.3 MB
Auxiliary	n new ints 28n + two lists 9n + 2 x 56	1.8 MB
Total	input + auxiliary	3.1 MB